



Manufacturing & Service Operations Management

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Maximum Profit Scheduling

Zhi-Long Chen, Nicholas G. Hall,

To cite this article:

Zhi-Long Chen, Nicholas G. Hall, (2008) Maximum Profit Scheduling. Manufacturing & Service Operations Management 10(1):84-107. <https://doi.org/10.1287/msom.1060.0144>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2008, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Maximum Profit Scheduling

Zhi-Long Chen

Smith School of Business, University of Maryland, College Park, Maryland 20742,
zchen@rsmith.umd.edu

Nicholas G. Hall

Fisher College of Business, The Ohio State University, Columbus, Ohio 43210,
hall_33@cob.osu.edu

The classical scheduling literature considers many problems where a given set of jobs must be processed at minimum cost, subject to various resource constraints. The literature only considers the issue of revenue generation in a very limited way, by allowing a job to remain unprocessed and its revenue contribution to be lost. By contrast, we consider three diverse practical situations where efficient scheduling affects revenue in much more general and realistic ways. First, we study two make-to-order environments where efficient scheduling increases customer goodwill, thus stimulating demand in different ways. Second, we study two make-to-stock environments where efficient scheduling creates inventory, thus also stimulating demand in different ways. Third, we study new product markets where efficient scheduling leads to a company becoming the first mover, and thus acquiring a larger market share. In each case, we provide both a computationally efficient algorithm for scheduling and a proof that a much more efficient algorithm is unlikely to exist. For both the make-to-stock and make-to-order problems, we also describe heuristic approaches that are easy to implement, and we study their average performance. The results show that substantial benefits arise from considering the implications of efficient scheduling for revenue and net profit. The practical impact of our work is to demonstrate the importance of efficient scheduling, not only in controlling cost, but also in increasing revenue and net profit.

Key words: manufacturing; scheduling; profit maximization; make-to-stock; make-to-order; algorithms; heuristics

History: Received: February 22, 2005; accepted: October 27, 2006. Published online in *Articles in Advance* December 11, 2007.

1. Introduction

The classical scheduling literature assumes that a set of available jobs, which represents customer demand, is given. The manufacturer processes those jobs at minimum cost, subject to resource constraints. Revenue is considered only by allowing a job to remain unprocessed and its revenue contribution to be lost. In this paper, we discuss several realistic situations where efficient scheduling stimulates customer demand and thus increases revenue. These situations suggest the need for a new direction in scheduling research that considers the implications for the revenue and net profit, as well as for the cost, of scheduling decisions.

Modeling the revenue implications of scheduling decisions is important where the following situations arise:

- In make-to-order systems, efficient delivery of an order generates customer goodwill that leads to future orders, as occurs frequently in many industries

(Christopher 1998, pp. 44–47; Stock and Lambert 2001, pp. 95–143). Thus, in a multiple-period planning problem, efficient scheduling during one time period may lead to improved customer service and increased demand in subsequent periods. Similarly, in a competitive contract bidding process, having an extensive inventory of a product that is similar to the one in the competition is evidence of reliable delivery. Zinn and Liu (2001) develop a model that identifies the long-run effects of customer service on goodwill, and thus on future demand.

- Also in make-to-order systems, manufacturers increasingly view short lead times as a significant and sustainable competitive advantage (Blackburn 1991). Thus, the influence of lead times on demand and the search for ways to reduce lead times by efficient scheduling are more important than ever (Palaka et al. 1998).

- In make-to-stock systems, potential customers are motivated by the presence of inventory to make

purchases. For example, in a traditional store, large amounts of inventory are more visible than small amounts (Balakrishnan et al. 2004), and this increased visibility stimulates demand. An overview of how this stimulation occurs is provided by Blattberg and Neslin (1990). Similarly, in an online store the availability of items, which is determined by the inventory level, is an important component of the purchase decision.

- Having products available in a new market enhances brand image and may also discourage competitors from entering the market, thereby stimulating long-run demand. For example, in emerging technology markets where the first-mover advantage is significant, early market presence often leads to market dominance (Lieberman and Montgomery 1988; Davis 1989; Kerin et al. 1992; Nahmias 1997, pp. 14–15).

The role of scheduling within the context of manufacturing planning is described by Nahmias (1997, pp. 398–400). Useful general references on scheduling include the books by Brucker (1998) and Pinedo (2002). All the scheduling problems discussed therein assume a given demand, and the only consideration of lost revenue is in problems where a job is allowed to remain unprocessed (Moore 1968). Therefore, those works do not consider the variety of ways in which scheduling decisions may affect demand and revenue in practice.

The fundamental distinction between make-to-order and make-to-stock manufacturing is discussed by Weiss and Gershon (1993). In make-to-order manufacturing, the processing of a job is initiated by a customer request that is specific to that job; whereas, in make-to-stock manufacturing, the processing of a job is initiated in response to a demand forecast. Most classical scheduling models assume a make-to-order environment, where the orders received define the demand at the scheduling stage. These models fail to consider the possibility that efficient scheduling and delivery during one time period may result in increased demand in later time periods. The make-to-stock environment naturally leads to situations where the presence of inventory stimulates demand, which is also not considered in classical models. We consider both make-to-order and make-to-stock manufacturing environments in this work.

In make-to-order environments, the widely adopted just-in-time manufacturing philosophy (Ohno 1988, Shingo 1989) emphasizes the importance of short lead times. Specifically, firms compete not only on price, but also on the lead times with which they deliver their products to customers. As a result, the demand for a product is a function of the lead time in many make-to-order systems. Models where customer demand is a function of lead time are studied by Wein (1991), Duenyas and Hopp (1995), and Spearman and Zhang (1999). Also, the effect of sharing lead-time information with customers on system performance measures such as waiting time, profit, and service level is studied by Whang (1988) and Whitt (1999). However, there are apparently no studies of scheduling decisions that arise in manufacturing systems with lead-time-dependent demand.

In make-to-stock environments, the role of inventory levels in stimulating demand is considered by Baker and Urban (1988a, b) and by Gerchak and Wang (1994). Geunes et al. (2004) extend the classical economic order quantity and production order quantity models to include market selection, which refocuses those models on profit maximization instead of cost minimization. Loparic et al. (2001) similarly generalize the classical uncapacitated lot-sizing problem by treating sales as a decision variable. Balakrishnan et al. (2004) formulate an inventory management problem as one of net profit maximization. They develop various lot-sizing models for which they characterize optimal policies. They conclude that the best policy to use is strongly dependent on how large the set-up costs are. These studies motivate our work because they suggest that scheduling decisions that increase inventory levels at various points in time will enhance demand. It follows that, analogously with the production planning models of Loparic et al. (2001) and Geunes et al. (2004), scheduling models should look beyond minimizing cost and attempt to maximize revenue or net profit.

The refocusing of scheduling research that we propose, away from cost models toward models that also consider revenue and net profit, parallels the recent refocusing of research from supply chain management toward demand chain management. As Selen and Soliman (2002, p. 667) point out, “the rapid uptake of technology, and in particular the Internet, has resulted

in the continual evolution of the demand chain concept with a shift away from supply chains towards demand chains and demand chain management.” Soliman and Youssef (2001) attribute this shift to a transfer of power from the supplier to the customer. Lee and Whang (2001) provide case studies of companies with exemplary demand chain management systems. Frohlich and Westbrook (2002) investigate the effectiveness of various strategies for implementing and supporting demand chain management.

Of course, it would be incorrect to assume that practicing managers who make scheduling decisions are unaware of the implications of those decisions for profit. Some rules of thumb for considering profit are already available. For example, managers can assign higher priorities to jobs that have larger impacts on profit. These higher priorities naturally lead many scheduling procedures to schedule those jobs earlier. However, such approaches fail to represent the detailed mechanisms by which efficient scheduling may stimulate demand and, therefore, frequently do not provide close-to-optimal solutions. Our paper provides the detailed planning tools necessary to make scheduling decisions that optimally or approximately maximize profit. We also provide computational validation of these tools by estimating the increased profit that they provide relative to the simplistic priority approaches discussed above.

Under several alternative assumptions about the way in which scheduling decisions affect demand, we describe mathematical models for maximizing net profit. It is important to note that our study of profit maximization differs from traditional throughput maximization (Seidmann and Tenenbaum 1994, Hall et al. 1997, Berman and Dasgupta 2000), because we consider demand to be a function of scheduling decisions.

This paper has six objectives, with the first four objectives focused on scheduling practice: Our first objective is to model several of the most important ways in which efficient scheduling can stimulate demand. By doing so, we will help practicing managers to understand the mechanisms by which this process occurs. Our second objective is to provide efficient optimal solution procedures that will enable practicing managers to maximize profit when making

scheduling decisions. However, these solution procedures are rather complicated and may require investment in technical expertise, software, and detailed implementation. Therefore, our third objective is to describe simpler heuristic procedures for our profit maximization models and to validate their performance computationally. These heuristic procedures are simple and fast enough to be implemented on a spreadsheet for practical size problems. Our fourth objective is to provide practical insights about the potential of scheduling as a planning tool for controlling revenue and cost. These insights include identifying problem parameters that have a significant impact on the performance of our approaches relative to traditional production-planning approaches and identifying solution characteristics such as capacity utilization that are changed by considering revenue.

Our last two objectives focus on research issues in scheduling. The fifth objective is to establish the limits of our analysis by identifying intractable problems among those studied. Our sixth and final objective is to identify important new directions for scheduling research. Our general perspective that efficient scheduling can help drive profitability suggests numerous new research issues and possibilities. By achieving these six objectives, we expect to convince practicing managers that efficient scheduling can lead to significant improvements in revenue and profit and, furthermore, to demonstrate how those improvements can be achieved. We also expect to stimulate research interest in scheduling problems where demand is a function of scheduling decisions.

In all the models we consider, there are k different job types to be scheduled on a single production line within a planning horizon $[0, T]$. We assume throughout the paper that k is fixed. This assumption applies to any practical situation where the set of available jobs is fixed. That is, customers can order an arbitrary quantity from a job type list of constant size. Our assumption excludes only job customizations at the customer's request. Each type j job, for $j = 1, \dots, k$, has a given processing time p_j . We define $p_{\min} = \min\{p_j \mid 1 \leq j \leq k\}$ and $p_{\max} = \max\{p_j \mid 1 \leq j \leq k\}$ and assume that $T \geq p_{\max}$. Demand for each job type is assumed to be deterministic and occurring only at discrete time points, $t = 1, \dots, T$. All data are assumed to be integer. Also, we assume that a first-in,

first-out (FIFO) rule is used for allocating production to demand. Thus, demand absorbs production of the units of job type j in the same order in which those units are produced. Additional notation and definitions are given below where necessary.

This paper is organized as follows. Section 2 considers two make-to-order manufacturing environments where demand is a function of previous manufacturing performance, as measured by lead time or by on-time performance. We provide algorithm and computational complexity results for the maximization of net profit. We also design and test simple heuristic approaches, and evaluate the improvement in profit provided by our approaches relative to traditional scheduling approaches. These traditional approaches, which we consider for the purposes of comparison, might reasonably be used by a scheduling manager who is aware that there is a connection between schedule efficiency and revenue, but who lacks the methodology provided by this paper. In §3, we similarly analyze two make-to-stock manufacturing environments where the demand at each time point is a function of the inventory level at that time or the average inventory up to that time. Section 4 contains an analysis of first-mover problems, where the revenue earned by delivering a particular job type to market decreases from one period to the next. Finally, §5 provides a conclusions and some suggestions for future research.

2. Make-to-Order Problems

In this section, we consider two make-to-order problems where processing of a job occurs only after a specific order for that job has been received, and a job once processed is delivered immediately to the customer. Thus, there is no inventory of finished jobs in the system. The demand for a job type is a deterministic function of the service level, as measured by delivery time performance, provided to that job type so far. This models the effect of customer goodwill on future sales. Section 2.1 describes optimal algorithms and demonstrates that much more efficient algorithms are unlikely to exist. Section 2.2 provides insights, heuristics, and comparisons to traditional scheduling approaches.

2.1. Optimal Schedules

In the first problem, MTO1, at the beginning of the planning horizon there are u_j^0 type j jobs that have been ordered. The demand for type j jobs at time $t = 1, \dots, T$ is given by $d_{jt}(q_j)$, for $j = 1, \dots, k$, a nonincreasing function of the average flow time q_j of all the type j jobs demanded by the customer up to time t , where q_j is calculated as follows. Let n_j and u_j denote the number of type j jobs that have been completed and the number of type j jobs that have been ordered but not completed by time t , respectively. Let C_j denote the sum of the completion times of all the completed type j jobs, and A_j the sum of the ordering times of all the type j jobs (both completed and uncompleted) that have been ordered by time t . Then, $q_j = (C_j + u_j t - A_j)/n_j$. Each type j job, once processed in response to a demand, earns a revenue r_j . Although there is no finished job-holding cost, there are work-in-process costs associated with the jobs that have been ordered but not yet completed. The work-in-process cost of type j jobs from time t to time $t + 1$ is defined as $g_j u_j$, where g_j is the per unit work-in-process cost of type j jobs, and u_j is the number of type j jobs that have been ordered but not completed by time t . The problem is to determine how many of each type of job to process and what schedule to use to maximize the total profit, defined as the total revenue minus the total work-in-process cost, during the planning horizon.

In Problem MTO1, we assume that all incoming orders are accepted. However, if there is not enough capacity to produce an order, then the revenue for this order is not earned. The acceptance of an order that is never produced may affect the computation of average flow time q_j , but this never affects total profit. Suppose, for example, that accepting this order reduces q_j and thus stimulates future demand; from the FIFO assumption, this additional demand will never be met. Alternatively, suppose that accepting this order increases q_j and thus reduces future demand; by assumption, the current order is not produced, and so from the FIFO assumption, the future demand is irrelevant.

We propose the following forward dynamic programming algorithm for Problem MTO1 with a fixed number of job types.

ALGORITHM DP-MTO1.

Value Function

$F(t; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k)$ = maximum total profit earned over the time period $[0, t]$, given that at time t a total of n_j type j jobs has been processed, and u_j type j jobs have been ordered but not processed, and that the total flow time of the type j jobs is f_j over the period $[0, t]$, for $j = 1, \dots, k$.

Boundary Condition

$$F(0; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k) = \begin{cases} 0, & \text{if } n_j = 0, u_j = d_{j0}(0), \text{ and } f_j = 0, \\ & \text{for } j = 1, \dots, k \\ -\infty, & \text{otherwise.} \end{cases}$$

Optimal Solution Value

$$\max_{n_1, \dots, n_k, u_1, \dots, u_k, f_1, \dots, f_k} \{F(T; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k)\}.$$

Recurrence Relation

$$F(t; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k) = \max \left\{ \begin{aligned} & F(t-1; n_1, \dots, n_k; u'_1, \dots, u'_k; f'_1, \dots, f'_k) - \sum_{j=1}^k g_j u'_j, \\ & \max_{1 \leq i \leq k: u_i^{t-p_i} \geq 1} \left\{ r_i + F(t-p_i; n_1, \dots, n_{i-1}, n_i-1, \right. \\ & \quad \left. n_{i+1}, \dots, n_k; u_1^{t-p_i}, \dots, u_k^{t-p_i}; \right. \\ & \quad \left. f_1^{t-p_i}, \dots, f_k^{t-p_i} \right\} - \sum_{j=1}^k \sum_{\tau=1}^{p_i} g_j u_j^{t-\tau} \end{aligned} \right\},$$

where $u'_j = u_j - d_{jt}(f_j/n_j)$, and $f'_j = f_j - (u_j - d_{jt}(f_j/n_j))$. For $h = 1, \dots, p_i$, u_j^{t-h} is the number of unprocessed type j jobs at time $t-h$, and f_j^{t-h} is the average flow time of the type j jobs during the period $[0, t-h]$. These values are calculated in the order $h = 0, \dots, p_i - 1$, using the following recursions

$$u_j^{t-h-1} = \begin{cases} u_j^{t-h} - d_{j,t-h}(f_j^{t-h}/n_j^{t-h}) + 1, & \text{if } j = i \text{ and } h = 0 \\ u_j^{t-h} - d_{j,t-h}(f_j^{t-h}/n_j^{t-h}), & \text{otherwise.} \end{cases}$$

$$f_j^{t-h-1} = f_j^{t-h} - [u_j^{t-h} - d_{j,t-h}(f_j^{t-h}/n_j^{t-h})],$$

for $j = 1, \dots, k$,

where

$$n_j^{t-h} = \begin{cases} n_j - 1, & \text{if } j = i \text{ and } h = 0 \\ n_j, & \text{otherwise.} \end{cases}$$

THEOREM 1. Algorithm DP-MTO1 finds an optimal schedule for Problem MTO1 in $O(kp_{\max} T^{4k+1}/p_{\min}^{3k})$ time.

PROOF. For a given state $(t; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k)$, the recurrence relation compares the cost of two possible state transitions. In the first, the machine schedules one unit of idle time from time $t-1$ to time t . In the second transition, a type i job is processed in the interval $[t-p_i, t]$. It follows that Algorithm DP-MTO1 compares the cost of all possible state transitions, and thus finds an optimal schedule.

It is sufficient to consider states $(t; n_1, \dots, n_k; u_1, \dots, u_k; f_1, \dots, f_k)$ with $t \leq T$, $n_j \leq T/p_{\min}$, $u_j \leq T/p_{\min}$, and $f_j \leq T^2/p_{\min}$. Hence, there are no more than $T^{4k+1}/p_{\min}^{3k}$ states in the dynamic program. To calculate the value for each state, $O(k)$ comparisons are made, using an $O(p_{\max})$ recursion. Therefore, the overall time complexity of Algorithm DP-MTO1 is $O(kp_{\max} T^{4k+1}/p_{\min}^{3k})$. $\square$

The input size of the problem is at least $O(kT^2)$, because there are kT demand functions, each with encoding length $O(T)$. Therefore, Theorem 1 implies that Algorithm DP-MTO1 runs in polynomial time for the problem with a fixed number of job types k . However, we now prove a negative result for MTO1 with an arbitrary number of job types:

THEOREM 2. The recognition version of Problem MTO1 with an arbitrary number of job types is unary NP-complete.

PROOF. The categorization “unary NP-complete” is equivalent to “NP-complete in the strong sense” (Papadimitriou and Steiglitz 1982). See the online appendix for the proof.¹ $\square$

We now consider a second make-to-order problem, MTO2, in which the demand for a job type at each time point is a nondecreasing function of the percentage of the jobs that have been completed by their lead times so far. More specifically, each job type j has a known lead time L_j . If an order for type j jobs is not

¹ Some proofs are contained in an online appendix to this paper available on the Manufacturing & Service Operations Management website (<http://msom.pubs.informs.org/ecompanion.html>).

satisfied within time L_j after the arrival of the order, then the order is assumed to be lost. At time 0, there are u_j^0 type j jobs that have been ordered. The demand for type j jobs at time $t \geq 1$ is given by the function $d_{jt}(\theta_j)$ which is a nondecreasing function of the percentage θ_j of type j jobs completed on time, where θ_j is calculated as follows, for $j = 1, \dots, k$. Let s_j and n_j denote the total number of type j jobs that have been ordered and the total number of type j jobs that have been completed by their lead times in the period $[0, t]$, respectively. Then, at time t , $\theta_j = n_j/s_j$. We assume that $\theta_j = 1$ if $n_j = s_j = 0$. Each type j job, once completed within its lead time in response to a demand, earns a revenue r_j . There is no work-in-process cost in Problem MTO2, which models practical situations where lead times are short, hence the work-in-process cost associated with an order is negligible compared to its revenue.

For the special case of Problem MTO2 where $L_j = p_j$, i.e., the lead time of a job is equal to the processing time of the job, we propose the following dynamic programming algorithm that runs in polynomial time for fixed k . In this problem, a job is either processed immediately after it is requested by the customer, or it is lost. One motivation for this assumption arises in the production of time-critical products. For example, toy manufacturers (Johnson 2001) often outsource the production of time-critical products (e.g., toys based on movie characters) to subcontractors who can start production immediately. A second motivation for this assumption arises in the scheduling of a dedicated machine that handles emergency or very-high-priority jobs that need to be processed without delay.

ALGORITHM DP-MTO2.

Value Function

$F(t; n_1, \dots, n_k; u_1, \dots, u_k)$ = maximum total revenue earned over the time period $[0, t]$, given that by time t a total of n_j type j jobs has been processed, and a total of u_j type j jobs has been lost, for $j = 1, \dots, k$.

Boundary Condition

$$F(0; n_1, \dots, n_k; u_1, \dots, u_k) = \begin{cases} 0, & \text{if } n_j = 0, \text{ and } u_j = 0, \text{ for } j = 1, \dots, k. \\ -\infty, & \text{otherwise.} \end{cases}$$

Optimal Solution Value

$$\max_{n_1, \dots, n_k, u_1, \dots, u_k} \{F(T; n_1, \dots, n_k; u_1, \dots, u_k)\}.$$

Recurrence Relation

$$F(t; n_1, \dots, n_k; u_1, \dots, u_k) = \max \left\{ \begin{aligned} &F(t-1; n_1, \dots, n_k; u'_1, \dots, u'_k), \\ &\max_{1 \leq i \leq k: d_{i,t-p_i}((n_i-1)/(n_i+u_i^{t-p_i}-1)) \geq 1} \{r_i + F(t-p_i; \\ &n_1, \dots, n_{i-1}, n_i-1, n_{i+1}, \dots, n_k; u_1^{t-p_i}, \dots, u_k^{t-p_i})\}, \end{aligned} \right\},$$

where $u'_j = u_j - d_{jt}(n_j/(n_j + u_j))$.

For $h = 1, \dots, p_i$, u_j^{t-h} is the number of lost type j jobs by time $t-h$. These parameters are calculated in the order $h = 0, \dots, p_i - 1$, using the following recursion

$$u_j^{t-h-1} = \begin{cases} u_j^{t-h} - d_{j,t-h}(n_j^{t-h}/(n_j^{t-h} + u_j^{t-h})) + 1, & \text{if } j = i \text{ and } h = p_i - 1 \\ u_j^{t-h} - d_{j,t-h}(n_j^{t-h}/(n_j^{t-h} + u_j^{t-h})), & \text{otherwise,} \end{cases}$$

where

$$n_j^{t-h-1} = \begin{cases} n_j^{t-h} - 1, & \text{if } j = i \text{ and } h = p_i - 1 \\ n_j^{t-h}, & \text{otherwise.} \end{cases}$$

THEOREM 3. Algorithm DP-MTO2 finds an optimal schedule for Problem MTO2 with $L_j = p_j$ in $O(kp_{\max} T^{2k+1}/p_{\min}^{2k})$ time.

PROOF. For a given state $(t; n_1, \dots, n_k; u_1, \dots, u_k)$, the recurrence relation compares the cost of the two possible state transitions. In the first, the machine schedules one unit of idle time from time $t-1$ to time t . In the second transition, a type i job is processed in the interval $[t-p_i, t]$. Because the costs of all possible state transitions are compared, Algorithm DP-MTO2 finds an optimal schedule.

It is sufficient to consider states $(t; n_1, \dots, n_k; u_1, \dots, u_k)$ with $t \leq T$, $n_j \leq T/p_{\min}$, and $u_j \leq T/p_{\min}$. Hence, there are no more than $T^{2k+1}/p_{\min}^{2k}$ states in the dynamic program. To calculate the value for each state, $O(kp_{\max})$ calculations are required. Therefore, the overall time complexity of Algorithm DP-MTO2 is $O(kp_{\max} T^{2k+1}/p_{\min}^{2k})$. $\square$

Since the input size of the problem is at least $O(kT)$, Theorem 3 implies that Algorithm DP-MTO2 runs in polynomial time for Problem MTO2 with $L_j = p_j$ and a fixed number of job types. However, we now prove a negative result for MTO2 with an arbitrary number of job types.

THEOREM 4. *The recognition version of Problem MTO2 with an arbitrary number of job types is unary NP-complete.*

PROOF. See the online appendix. $\square$

2.2. Insights, Heuristics, and Comparisons

The traditional production planning approach in make-to-order environments is to schedule efficiently the orders that have arrived, without considering the impact of scheduling on future demand (Nahmias 1997). However, we evaluate the improvement in profit that is provided by algorithms that take into account the impact of scheduling on future demand. Although demand occurs at any point in time in the general Problems MTO1 and MTO2, in practice, orders are typically placed periodically because there is a fixed cost (Nahmias 1997) associated with each order. Thus, we assume that the planning horizon $[0, T]$ is divided by $m + 1$ review points $R_0, \dots, R_m$, such that $R_i = i\tau$ for $i = 0, \dots, m$, for some $\tau > 0$, and $T = R_m$, where $d_{jt}(\cdot) = 0$ if $t \notin \{R_0, \dots, R_m\}$, for $j = 1, \dots, k$. Clearly, Algorithms DP-MTO1 and DP-MTO2 given in §2.1 that solve the Problem MTO1, and the Problem MTO2 with $L_j = p_j$, respectively, can solve those problems under this assumption.

To provide a comparison between the traditional and proposed approaches that is not confounded by capacity issues, we make the following assumption.

ASSUMPTION 1. *The production capacity in each time interval $[R_i, R_{i+1}]$, for $i = 0, \dots, m - 1$, is sufficient that, using any of the approaches compared here, all the demand which occurs at R_i , if not lost due to the deadline, can be satisfied in that interval alone.*

It should be noted that Assumption 1 implies that there is no lost demand in any of the approaches we use for Problem MTO1. However, there may be lost demand for Problem MTO2 because the jobs have deadlines in this problem. Under Assumption 1, the traditional production-planning approaches for Problems MTO1 and MTO2, denoted by TR-MTO1 and TR-MTO2, respectively, can be described as follows.

ALGORITHM TR-MTO1.

In each time period $[R_{i-1}, R_i]$, for $i = 1, \dots, m$, do the following:

Step 0. Let a_{ij} denote the number of type j jobs that are ordered at review point R_{i-1} , where $a_{1j} = u_j^0$ and

$a_{ij} = d_{j, R_{i-1}}(f_j)$ for $i \geq 2$, where f_j is the average flow time of type j jobs over the time interval $[0, R_{i-1}]$.

Step 1. Schedule all the a_{ij} type j jobs, for $j = 1, \dots, k$, in this time period to minimize the total work-in-process cost.

ALGORITHM TR-MTO2.

In each time period $[R_{i-1}, R_i]$, for $i = 1, \dots, m$, do the following:

Step 0. Let a_{ij} denote the number of type j jobs that are ordered at review point R_{i-1} , where $a_{1j} = u_j^0$ and $a_{ij} = d_{j, R_{i-1}}(\theta_j)$ for $i \geq 2$, where θ_j is the percentage of type j jobs completed on time over the time interval $[0, R_{i-1}]$.

Step 1. Schedule a subset of the a_{ij} type j jobs on time, for $j = 1, \dots, k$, in this review period to maximize the total revenue of the scheduled jobs.

In Step 1 of TR-MTO1, the total work-in-process cost is minimized by processing the jobs in nondecreasing order of p_j/g_j without inserted idle time (Smith 1956). The scheduling problem in Step 1 of TR-MTO2 is the single machine weighted number of late jobs problem that is binary NP-complete, but can be solved by the pseudopolynomial time dynamic programming algorithm of Lawler and Moore (1969). Under Assumption 1, both Algorithms TR-MTO1 and TR-MTO2 generate optimal solutions for Problems MTO1 and MTO2, respectively, if the impact of scheduling decisions on demand is not considered.

To evaluate TR-MTO1 and TR-MTO2, we propose simple greedy heuristics, denoted by GH-MTO1 and GH-MTO2 for Problems MTO1 and MTO2, respectively, that consider the fact that demand is a function of scheduling efficiency. These heuristics use a greedy rule to choose a job type and a number of jobs of this type to process next, such that the total profit or revenue is maximized if the chosen jobs are processed next.

HEURISTIC GH-MTO1.

In each time interval $[R_{i-1}, R_i]$, for $i = 1, \dots, m$, do the following:

Step 0. See Step 0 of TR-MTO1.

Step 1. Let t denote the current time. Initially, $t = R_{i-1}$. Let Q denote the set of job types that have not been scheduled. For $j \in Q$, let E_j denote the total profit

of the partial schedule over the time period $[t, T]$, where

(i) starting from time t in the current time interval, the a_{ij} type j jobs are scheduled first, followed by the other types of jobs of Q in nondecreasing order of p_j/g_j ; and

(ii) in each of the later time intervals, $[R_s, R_{s+1}]$, for $s = i, \dots, m-1$, all the jobs that have arrived at time R_s are scheduled in nondecreasing order of p_j/g_j .

Step 2. Choose job type l such that $E_l = \max\{E_j \mid j \in Q\}$. Schedule all the a_{il} type l jobs in the interval $[t, t + a_{il}p_l]$, and set $t = t + a_{il}p_l$, and $Q = Q \setminus \{l\}$. If $Q = \emptyset$, then move to the next time interval, set $i = i + 1$, and go to Step 0. Otherwise, go to Step 1.

HEURISTIC GH-MTO2.

In each time interval $[R_{i-1}, R_i]$, for $i = 1, \dots, m$, do the following:

Step 0. See Step 0 of TR-MTO2.

Step 1. Let t denote the current time. Initially, $t = R_{i-1}$. Let Q denote the set of job types that have not been scheduled, where the job types are sorted in nondecreasing order of L_j . Let u be the first job type in Q , and z_u be the largest integer such that $z_u \leq a_{iu}$ and $t + z_u p_u - R_{i-1} \leq L_u$. For $z = 0, \dots, z_u$, let E_z denote the total revenue of the on-time jobs in the partial schedule over the time period $[t, T]$, where

(i) starting from time t in the current time interval, z type u jobs are scheduled first, followed by a subset of the jobs of the other types in Q that maximizes the total revenue of the on-time jobs;

(ii) in each of the later time intervals, $[R_s, R_{s+1}]$, for $s = i, \dots, m-1$, a subset of the jobs that have arrived at time R_s , which maximizes the total revenue of the on-time jobs is scheduled.

Step 2. Choose y such that $E_y = \max\{E_z \mid z = 0, \dots, z_u\}$. Schedule y type u jobs in the interval $[t, t + yp_u]$, and set $t = t + yp_u$, and $Q = Q \setminus \{u\}$. If $Q = \emptyset$, then move to the next review point, set $i = i + 1$, and go to Step 0. Otherwise, go to Step 1.

In Step 1 of both heuristics, to compute E_j or E_z , the jobs are scheduled in the time period $[R_i, T]$ in exactly the same way as in TR-MTO1 or TR-MTO2, respectively.

Our computational experiments address the following questions: (i) How much can the total profit (respectively, total revenue) generated by TR-MTO1

(respectively, TR-MTO2) be improved by using GH-MTO1 (respectively, GH-MTO2)? (ii) How does this improvement vary with problem parameters?

We first report results on Problem MTO1. We use the following demand function for job type $j = 1, \dots, k$, and time $t \in \{R_1, \dots, R_m\}$: $d_{jt}(f_j) = \lceil \alpha_{jt} f_j^{-\beta_{jt}} \rceil$, where $\alpha_{jt}, \beta_{jt} > 0$. Preliminary testing showed that only parameters k, m, r_j, α_{jt} , and β_{jt} have a significant impact on the relative improvement between the solutions generated by TR-MTO1 and GH-MTO1. Therefore, for comparing these two algorithms, we only report results on test problems with ranges for these five parameters. Our parameter settings follow:

(a) The number of job types $k \in \{4, 8, 16\}$, number of review points $m \in \{3, 6, 12\}$, and the length of each review period $\tau = 8,000$.

(b) Processing times of jobs p_j and unit work-in-process cost g_j are both integers from $U[1, \dots, 10]$.

(c) Unit job revenues r_j are integers from either $U[2,500, \dots, 3,500]$ or $U[1,500, \dots, 4,500]$ or $U[500, \dots, 5,500]$.

(d) The initial numbers of jobs at time 0, u_j^0 , are integers from $U[1, \dots, 10]$.

(e) Demand function parameters α_{jt} are generated from $U[50, 100]$, $U[100, 200]$, or $U[200, 400]$, and β_{jt} are generated from $U[0.3, 0.8]$, $U[0.8, 1.3]$, or $U[1.3, 2.0]$.

We now explain our experimental design. The value of τ is set so that the capacity in each review period is sufficient, following Assumption 1. Preliminary testing showed that the variance of r_j in a test problem has a more significant impact on the relative improvement than the mean value of r_j . Thus, we use nested but different sizes of intervals with the same mean (3,000) to generate r_j . Also, the three intervals used for generating β_{jt} represent the case where the rate at which demand decreases with average flow time is either sublinear, or a mixture of sublinear and superlinear, or superlinear, respectively.

For each combination of the five parameters $(k, m, r_j, \alpha_{jt}, \beta_{jt})$, we test 10 randomly generated problem instances, for a total of $3 \times 3 \times 3 \times 3 \times 3 \times 10 = 2,430$ instances. Each row in Table 1 shows mean results over 90 random test problems: 10 for each of the nine combinations of $(\alpha_{jt}, \beta_{jt})$ with the same (k, m, r_j) combination. Each entry in the table is the mean relative percentage improvement from TR-MTO1 to

GH-MTO1 over the 10 random test problems, where the relative improvement from TR-MTO1 to GH-MTO1 on a given test problem is defined by $(F_{GH1} - F_{TR1})/F_{TR1}$, where F_{TR1} and F_{GH1} denote the total profit in a solution generated by TR-MTO1 and GH-MTO1, respectively.

The results in Table 1 show that the greedy heuristic GH-MTO1 provides a 29.9% improvement, on average, over the traditional approach TR-MTO1. Furthermore, the relative improvement over TR-MTO1 provided by GH-MTO1 typically increases with the variance of revenues of jobs r_j , and with parameters α_{jt} and β_{jt} in the demand function, decreases with the length of the planning horizon, and does not vary significantly with the number of job types k . The increase in relative improvement with r_j is predictable, since our model optimizes the profit component, whereas the traditional model does not. Similarly, larger values of α_{jt} and β_{jt} define a more sensitive demand function, which again favors the profit-maximization approach. Note that GH-MTO1 only considers revenue in detail in the current period; therefore, the more periods there are the smaller the relative improvement over TR-MTO1.

In addition, we find that, in the solutions generated by GH-MTO1, the job sequences in most time intervals lie between nonincreasing order of g_j/p_j (which is optimal if only work-in-process cost is considered) and nonincreasing order of r_j/p_j (which is a good sequence if only revenue is considered). This implies that in an environment where demand is a function of the flow time of the completed jobs, even if the exact demand functions are not known, using a job sequence that considers both g_j/p_j and r_j/p_j (e.g., nonincreasing order of a convex combination of g_j/p_j and r_j/p_j) is likely to generate better solutions than using nonincreasing order of g_j/p_j , as in the traditional approach.

Based on this observation, we conducted another set of tests on a new strategy, which is denoted by TR-MTO1-G. This strategy generalizes the traditional approach TR-MTO1, which completely ignores revenue effects, by considering to some extent the impact of the current schedule on future revenue. We therefore propose TR-MTO1-G as a reasonable benchmark that might be used by managers who recognize the revenue-generating effect of efficient scheduling. The

generalized approach TR-MTO1-G has the same steps as TR-MTO1, except that in Step 1 the jobs are scheduled in nonincreasing order of the ratio $(r_j + \lambda g_j)/p_j$, where $\lambda = \sum_{j=1}^k r_j / \sum_{j=1}^k g_j$ is a scaling parameter that ensures that in the ratio $(r_j + \lambda g_j)/p_j$ an equal weight is placed on revenue and cost. We expect that a customized choice of λ would provide better results for heuristic TR-MTO1-G, but we have not explored this possibility, since the heuristic is used here only as a benchmark. Table 2 shows the relative percentage improvement offered by the generalized approach TR-MTO1-G over TR-MTO1 based on the same test problems as for Table 1. Although TR-MTO1-G is a simple modification of the traditional approach TR-MTO1, our test shows that this strategy improves the total profit by an average of 8.7% for Problem MTO1. However, this improvement is much smaller than the 29.9% average improvement found for GH-MTO1 in Table 1.

For Problem MTO2, we use the following demand function: $d_{jt}(\theta_j) = \lceil D_{jt} \theta_j^{\beta_{jt}} \rceil$, for $j = 1, \dots, k$ and $t \in \{R_1, \dots, R_m\}$, where $D_{jt} \geq 1$ is a constant representing the maximum possible demand at time t , and $\beta_{jt} > 0$. We compare approaches TR-MTO2 and GH-MTO2 computationally on a special case of Problem MTO2 with $L_j = \gamma p_j$ for some constant $\gamma \geq 1$ common to all job types. Since our preliminary testing showed that the relative improvement from TR-MTO2 to GH-MTO2 is mainly sensitive to the four parameters k, m, γ , and β_{jt} , we vary these parameters. Our parameter settings are

- (a) the number of job types $k \in \{4, 8, 16\}$, number of review points $m \in \{3, 6, 12\}$, and the length of each review period $\tau = 100$ (which is set so that Assumption 1 is satisfied);
- (b) processing times of jobs p_j and unit revenues r_j , both integers from $U[1, \dots, 10]$;
- (c) the lead time parameter $\gamma \in \{1, 2, 3\}$;
- (d) initial number of jobs at time 0, u_j^0 , integers from $U[0, \dots, 6]$;
- (e) demand function parameters D_{jt} , generated from $U[1, \gamma]$, and β_{jt} , generated from either $U[0.3, 0.8]$ or $U[0.8, 1.3]$ or $U[1.3, 2.0]$.

For each combination of the four parameters (k, m, γ, β_{jt}), we test 10 randomly generated problem instances, for a total of $3 \times 3 \times 3 \times 3 \times 10 = 810$ instances. Each row in Table 3 shows mean results

Table 1 Comparison of Approaches TR-MTO1 and GH-MTO1 for Problem MTO1

k	m	r_j	$\alpha_{jt} \in [50, 100] (\%)$			$\alpha_{jt} \in [100, 200] (\%)$			$\alpha_{jt} \in [200, 500] (\%)$		
			$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$	$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$	$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$
4	3	[2,500, 3,500]	12.1	21.8	26.3	15.6	27.7	39.7	35.0	33.9	49.2
		[1,500, 4,500]	12.4	23.0	26.2	16.2	29.4	39.7	41.3	36.3	50.7
		[500, 5,500]	14.5	25.3	27.1	19.7	32.9	40.7	50.3	40.1	54.3
4	6	[2,500, 3,500]	8.9	18.4	17.3	13.1	24.5	28.5	27.2	31.9	39.3
		[1,500, 4,500]	9.8	20.3	18.2	14.8	27.2	29.9	30.2	34.8	40.4
		[500, 5,500]	11.6	22.9	19.8	17.9	30.9	31.8	39.2	40.7	44.2
4	12	[2,500, 3,500]	7.4	15.4	15.0	10.2	22.6	26.6	18.2	36.4	40.9
		[1,500, 4,500]	8.1	17.6	16.0	11.2	24.2	28.0	18.9	37.4	41.6
		[500, 5,500]	10.2	19.5	17.8	13.7	27.0	29.8	23.5	40.9	44.1
8	3	[2,500, 3,500]	11.8	17.2	16.7	18.2	22.8	26.6	35.1	30.9	34.7
		[1,500, 4,500]	15.3	20.0	18.5	25.6	28.8	30.3	39.7	37.8	41.1
		[500, 5,500]	22.1	26.9	21.8	41.3	40.3	38.0	58.6	57.3	55.9
8	6	[2,500, 3,500]	9.3	18.0	15.8	12.6	24.3	25.7	38.8	34.3	35.9
		[1,500, 4,500]	10.6	20.6	14.3	15.4	27.9	28.4	48.2	42.9	39.9
		[500, 5,500]	14.3	23.4	17.4	22.4	35.9	31.2	63.1	56.6	50.8
8	12	[2,500, 3,500]	7.6	14.6	10.1	11.7	20.4	23.7	27.0	34.1	33.9
		[1,500, 4,500]	9.0	16.8	11.1	13.4	22.4	22.8	32.6	39.2	39.6
		[500, 5,500]	12.1	17.9	12.9	117.7	27.0	26.8	48.7	46.9	47.7
16	3	[2,500, 3,500]	13.3	20.0	18.9	48.5	27.8	32.4	50.0	38.1	48.0
		[1,500, 4,500]	18.4	22.3	21.1	87.9	34.6	35.2	58.4	53.4	56.9
		[500, 5,500]	28.0	28.6	24.1	58.6	48.2	43.1	101.3	89.9	75.1
16	6	[2,500, 3,500]	8.7	16.8	16.5	15.4	24.8	29.0	59.2	34.8	44.4
		[1,500, 4,500]	11.8	21.8	19.1	20.1	31.7	32.4	88.8	45.4	50.3
		[500, 5,500]	15.7	28.8	23.0	28.3	40.4	38.7	114.7	57.5	61.0
16	12	[2,500, 3,500]	6.7	12.7	11.7	10.8	19.4	20.2	56.8	30.8	33.0
		[1,500, 4,500]	9.3	15.1	13.4	13.9	23.1	19.6	37.0	36.1	37.2
		[500, 5,500]	13.4	17.5	15.9	20.9	28.7	24.3	40.2	44.3	43.8

over 30 random test problems, 10 for each of the three cases of β_{jt} with the same (k, m, γ) combination. Each entry in the table is the mean relative percentage improvement from TR-MTO2 to GH-MTO2 over the 10 random test instances where the relative improvement is defined by $(F_{GH2} - F_{TR2})/F_{TR2}$, where F_{TR2} and F_{GH2} denote the total profit in a solution generated by TR-MTO2 and GH-MTO2, respectively.

The results in Table 3 show that Heuristic GH-MTO2 provides a mean improvement of 42.5% over the traditional approach TR-MTO2. This improvement is much larger when the lead time is short, e.g., $\gamma \in \{1, 2\}$, than when $\gamma \geq 3$. This is because as the lead time increases, the constraints in the problem become more relaxed. As a result, precise scheduling of the jobs becomes less important. The relative improvement over TR-MTO2 provided by GH-MTO2 typically increases with the parameter β_{jt} in the demand function. It does not vary systematically

with the number of job types k or the number of review points m . The increase in relative improvement as β_{jt} increases is once again because larger values of β_{jt} emphasize the revenue component in our profit-maximization model, unlike in the traditional model.

3. Make-to-Stock Problems

In this section, we consider two make-to-stock problems in which production occurs in anticipation of a demand, and a demand is satisfied immediately if sufficient inventory is available; otherwise, the demand is lost. The demand for a job type is a deterministic function of the inventory level of that job type (measured differently in the two problems studied). Section 3.1 describes optimal algorithms and demonstrates that much more efficient algorithms are unlikely to exist. Section 3.2 provides insights, heuristics, and comparisons to traditional scheduling approaches.

Table 2 Comparison of Approaches TR-MT01 and TR-MT01-G for Problem MT01

k	m	r_j	$\alpha_{jt} \in [50, 100] (\%)$			$\alpha_{jt} \in [100, 200] (\%)$			$\alpha_{jt} \in [200, 500] (\%)$		
			$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$	$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$	$\beta_{jt} \in [0.3, 0.8]$	$\beta_{jt} \in [0.8, 1.3]$	$\beta_{jt} \in [1.3, 2]$
4	3	[2,500, 3,500]	4.7	6.3	5.8	5.7	8.3	7.4	8.4	11.3	11.9
		[1,500, 4,500]	6.1	8.4	6.9	7.7	10.7	9.3	12.4	14.4	14.4
		[500, 5,500]	7.9	10.6	8.1	10.6	13.8	11.3	24.8	18.9	17.9
4	6	[2,500, 3,500]	4.5	8.3	8.5	5.1	8.4	9.9	4.6	7.8	9.8
		[1,500, 4,500]	5.9	10.3	9.7	7.0	10.7	11.4	8.6	10.6	12.0
		[500, 5,500]	7.6	13.0	11.3	9.8	13.7	13.6	18.1	14.3	15.0
4	12	[2,500, 3,500]	4.7	8.5	9.7	5.1	6.9	10.4	6.0	5.3	7.6
		[1,500, 4,500]	5.9	10.6	11.4	6.5	8.6	12.7	8.1	6.5	9.2
		[500, 5,500]	7.6	13.2	13.3	8.7	11.0	15.1	12.4	8.7	11.4
8	3	[2,500, 3,500]	5.1	5.6	3.2	6.2	7.1	5.1	10.8	8.7	6.9
		[1,500, 4,500]	6.4	6.5	4.5	9.4	9.5	6.7	19.4	12.0	9.5
		[500, 5,500]	9.1	8.8	6.2	14.4	12.5	8.9	24.2	17.1	13.0
8	6	[2,500, 3,500]	4.4	6.7	4.1	4.8	6.7	5.4	10.4	6.8	7.1
		[1,500, 4,500]	5.5	8.5	5.6	5.8	8.6	7.7	6.8	8.9	10.1
		[500, 5,500]	7.2	11.1	6.8	7.8	11.5	9.8	5.0	12.3	13.2
8	12	[2,500, 3,500]	5.1	8.7	5.5	5.7	7.1	7.1	7.8	7.0	6.6
		[1,500, 4,500]	6.4	10.5	6.6	7.4	9.1	9.1	10.2	9.6	9.5
		[500, 5,500]	8.5	13.5	8.0	9.5	12.1	12.0	12.7	12.8	13.5
16	3	[2,500, 3,500]	2.5	2.5	7.9	2.1	3.8	4.6	19.1	2.0	7.3
		[1,500, 4,500]	6.6	6.5	6.7	20.5	10.0	12.4	9.0	16.0	21.5
		[500, 5,500]	11.9	10.6	9.6	16.0	16.9	18.2	16.3	29.2	33.6
16	6	[2,500, 3,500]	2.4	2.3	1.3	2.8	2.8	2.1	3.6	3.1	9.7
		[1,500, 4,500]	5.3	6.1	4.7	6.6	7.8	9.1	6.1	11.2	16.4
		[500, 5,500]	9.3	11.3	7.4	11.8	13.9	13.5	32.2	20.1	25.0
16	12	[2,500, 3,500]	2.5	2.9	3.9	2.7	2.9	6.4	4.4	2.4	8.9
		[1,500, 4,500]	4.8	4.7	1.3	5.3	4.3	2.4	7.2	5.2	5.4
		[500, 5,500]	8.2	7.9	2.8	10.0	8.3	4.2	10.7	9.8	9.0

3.1. Optimal Schedules

In the first problem, MTS1, the demand for type j jobs at time t is a function $d_{jt}(I_j)$ of the current inventory level I_j of type j jobs, for $j = 1, \dots, k$ and $t = 1, \dots, T$. The demand function $d_{jt}(I_j)$ is nondecreasing in I_j for any t . We assume that a demand $d_{jt}(I_j)$ that exceeds the current inventory level results in lost sales of $d_{jt}(I_j) - I_j$. Therefore, the effective demand at time t is $\min\{d_{jt}(I_j), I_j\}$. At time 0, there is an inventory I_j^0 of type j jobs in the system. A type j job, when sold to satisfy a demand, earns a revenue r_j . If it is not sold immediately after it is processed, it incurs an inventory holding cost h_j per unit time. At time T , each unsold type j job has a salvage value s_j . The objective is to find a schedule that maximizes the total profit, which is defined as the total revenue from selling jobs minus the total inventory cost plus the total salvage value of unsold jobs at time T .

We propose a dynamic programming algorithm for Problem MTS1 with a fixed number of job types and arbitrary demand functions.

ALGORITHM DP-MTS1.

Value Function

$F(t, I_1, \dots, I_k)$ = maximum total net profit earned over the time period $[t, T]$, given that at time t there are I_j jobs of type j in inventory (after the demand for type j jobs has been satisfied or lost), for $j = 1, \dots, k$, and no job is being processed.

Boundary Condition

$$F(T, I_1, \dots, I_k) = \sum_{j=1}^k s_j I_j.$$

Optimal Solution Value

$$F(0, I_1^0, \dots, I_k^0).$$

Table 3 Comparison of Approaches TR-MT02 and GH-MT02 for Problem MT02

k	m	γ	$\beta_{jt} \in [0.3, 0.8]$ (%)	$\beta_{jt} \in [0.8, 1.3]$ (%)	$\beta_{jt} \in [1.3, 2]$ (%)
4	3	1	28.9	32.3	31.1
		2	8.7	12.7	24.1
		3	4.6	3.2	10.0
4	6	1	28.9	93.0	100.5
		2	6.5	18.0	42.4
		3	4.3	3.2	12.8
4	12	1	41.6	206.2	232.6
		2	8.4	10.5	51.1
		3	6.7	6.5	5.9
8	3	1	19.8	34.1	35.0
		2	5.3	10.6	14.3
		3	2.9	6.9	10.8
8	6	1	32.1	97.1	102.8
		2	7.7	20.5	40.5
		3	4.8	6.6	12.4
8	12	1	52.4	182.6	231.6
		2	5.9	13.8	64.6
		3	5.9	7.4	14.8
16	3	1	38.1	47.7	49.8
		2	11.4	20.8	21.1
		3	4.2	12.2	15.6
16	6	1	72.8	104.4	124.4
		2	10.2	29.1	45.7
		3	6.1	11.8	24.6
16	12	1	47.3	254.0	264.4
		2	10.5	40.9	83.5
		3	6.4	10.4	22.9

Recurrence Relation

$$F(t, I_1, \dots, I_k) = \max\{G_1, G_{21}, \dots, G_{2k}\}, \quad \text{where}$$

$$G_1 = F(t+1, \max\{0, I_1 - d_{1,t+1}(I_1)\}, \dots, \max\{0, I_k - d_{k,t+1}(I_k)\}) \\ + \sum_{j=1}^k r_j \min\{I_j, d_{j,t+1}(I_j)\} - \sum_{j=1}^k h_j I_j,$$

and for $i = 1, \dots, k$,

$$G_{2i} = F(t + p_i, x_{1,p_i} + \delta_{1i}, \dots, x_{k,p_i} + \delta_{ki}) + \sum_{j \neq i}^k r_j (I_j - x_{j,p_i}) \\ + r_i (I_i - x_{i,p_i} + 1) - \sum_{j=1}^k h_j \left(I_j + \sum_{h=1}^{p_i-1} x_{jh} \right),$$

where x_{jh} denotes the inventory of job type j at time $t + h$ (after the demand for type j jobs has been satisfied or lost), which is calculated as $x_{j1} = \max\{0, I_j -$

$d_{j,t+1}(I_j)\}$, $x_{jh} = \max\{0, x_{j,h-1} - d_{j,t+h}(x_{j,h-1})\}$, for $h = 2, \dots, p_i$, and $\delta_{ji} = 1$ if $i = j$ and $\delta_{ij} = 0$ otherwise.

THEOREM 5. Algorithm DP-MTS1 finds an optimal schedule for Problem MTS1 in $O(kp_{\max} T^{k+1}/p_{\min}^k)$ time.

PROOF. Algorithm DP-MTS1 is a backward dynamic program. The recurrence relation compares $k + 1$ options at time t : inserting one period of idle time (G_1), or starting to process a type i job (G_{2i}), for $i = 1, \dots, k$. In the case of processing a type i job, it is sufficient to check the inventory in the system when this job completes processing, i.e., at time $t + p_i$. This is because no new jobs are added to inventory during the time interval $(t, t + p_i)$. At time $t + p_i$, revenue $r_j(I_j - x_{j,p_i})$ is earned from the sale of type j jobs. Also, a holding cost of $h_j x_{jh}$ is incurred at time $t + h$. Therefore, the recurrence relation compares the cost of all possible state transitions and finds an optimal schedule. There are $O(T^{k+1}/p_{\min}^k)$ states in the dynamic program, and the value of each state is found in $O(kp_{\max})$ time, using the recursive equations for x_{jh} . Thus, the overall time complexity of Algorithm DP-MTS1 is $O(kp_{\max} T^{k+1}/p_{\min}^k)$. $\square$

The input size of the problem is at least $O(kT)$, because there are k demand functions, each with encoding length $O(T)$. This, along with the fact that $p_{\max} \leq T$, means that Algorithm DP-MTS1 runs in polynomial time when the number of job types k is fixed. However, we now prove a negative result for Problem MTS1 with an arbitrary number of job types, even where the demand functions are time-invariant.

THEOREM 6. The recognition version of Problem MTS1 with an arbitrary number of job types and time-invariant demand functions is unary NP-complete.

PROOF. See the online appendix. $\square$

We now consider a second make-to-stock problem, MTS2. Except for the demand function, all the parameters and the objective of this problem are the same as in the first problem. Let x_τ be the inventory of type j jobs at time τ before the demand and the newly completed jobs at that time are taken into account. The demand for type j jobs at time t , for $j = 1, \dots, k$ and $t = 1, \dots, T$, is given by the function $d_{jt}(A_j)$, where A_j is the average inventory of type j jobs over the time interval $[0, t]$ and is defined as $A_j = V_j/t$, where $V_j = \sum_{\tau=1}^t x_\tau$ denotes the total inventory of type j jobs

over the period $[0, t]$. The demand function $d_{jt}(A_j)$ is nondecreasing in A_j , for any $t = 1, \dots, T$.

We propose a backward dynamic programming algorithm for Problem MTS2, with a fixed number of job types and arbitrary demand functions.

ALGORITHM DP-MTS2.

Value Function

$F(t; I_1, \dots, I_k; V_1, \dots, V_k)$ = maximum total profit earned over the time period $[t, T]$, given that the inventory of type j jobs is I_j at time t and the total inventory of type j jobs over the time period $[0, t]$ is V_j (after the demand for type j jobs at time t is already satisfied or lost), for $j = 1, \dots, k$, and no job is being processed at time t .

Boundary Condition

$$F(T; I_1, \dots, I_k; V_1, \dots, V_k) = \sum_{j=1}^k s_j I_j.$$

Optimal Solution Value

$$F(0; I_1^0, \dots, I_k^0; 0, \dots, 0).$$

Recurrence Relation

$$F(t; I_1, \dots, I_k; V_1, \dots, V_k) = \max\{G_1, G_{21}, \dots, G_{2k}\},$$

where

$$G_1 = F(t+1; I'_1, \dots, I'_k; V'_1, \dots, V'_k) + \sum_{j=1}^k r_j \min\{I_j, d_{j,t+1}(V'_j/(t+1))\} - \sum_{j=1}^k h_j I_j,$$

where

$$I'_j = \max\{0, I_j - d_{j,t+1}(V'_j/(t+1))\} \quad \text{and} \quad V'_j = V_j t + I_j, \\ \text{for } j = 1, \dots, k.$$

For $i = 1, \dots, k$,

$$G_{2i} = F(t+p_i; I_1^{t+p_i}, \dots, I_k^{t+p_i}; V_1^{t+p_i}, \dots, V_k^{t+p_i}) + \sum_{j \neq i}^k r_j (I_j - I_j^{t+p_i}) + r_i (I_i - I_i^{t+p_i} + 1) - \sum_{j=1}^k h_j \left(\sum_{u=0}^{p_i-1} I_j^{t+u} \right),$$

where I_j^{t+u} and V_j^{t+u} , for $u = 1, \dots, p_i$ are the inventory of type j jobs at time $t+u$ and the average inventory of type j jobs over $[0, t+u]$, respectively. These

values can be computed recursively as follows. For $u = 1, \dots, p_i - 1$,

$$I_j^{t+u} = \max\{0, I_j^{t+u-1} - d_{j,t+u}(V_j^{t+u}/(t+u))\}, \\ \text{for } j = 1, \dots, k$$

$$V_j^{t+u} = V_j^{t+u-1}(t+u-1) + I_j^{t+u-1}, \quad \text{for } j = 1, \dots, k.$$

For $u = p_i$,

$$I_i^{t+u} = \max\{0, I_i^{t+u-1} - d_{i,t+u}(V_i^{t+u}/(t+u))\} + 1, \\ I_j^{t+u} = \max\{0, I_j^{t+u-1} - d_{j,t+u}(V_j^{t+u}/(t+u))\}, \quad \text{for } j \neq i \\ V_j^{t+u} = V_j^{t+u-1}(t+u-1) + I_j^{t+u-1}, \quad \text{for } j = 1, \dots, k.$$

THEOREM 7. Algorithm DP-MTS2 finds an optimal schedule for Problem MTS2 in $O(kp_{\max} T^{3k+1}/p_{\min}^{2k})$ time.

PROOF. Similar to the proof of Theorem 5. $\square$

Since the input size is at least $O(kT)$, and $p_{\max} \leq T$, Algorithm DP-MTS2 runs in polynomial time for fixed k . However, we now prove a negative result for Problem MTS2 with an arbitrary number of job types and time-invariant demand functions.

THEOREM 8. The recognition version of Problem MTS2 with an arbitrary number of job types and time-invariant demand functions is unary NP-complete.

PROOF. See the online appendix. $\square$

Both Algorithms DP-MTS1 and DP-MTS2 can easily be generalized to model a loss of goodwill that occurs from lost sales. For example, we can assume a loss-of-goodwill function that depends on the total lost sales of each job type up to that point in time. Then, we can add state variables $Y_1, \dots, Y_k$, which represent the total lost sales of job types $1, \dots, k$, to the value function. These state variables are updated in the recurrence relation whenever $d_{jt}(A_j) > I_j$. Also, we have assumed that a demand is satisfied immediately if sufficient inventory is available. These algorithms can easily be adapted to solve the case where a demand does not have to be satisfied even if sufficient inventory is available.

3.2. Insights, Heuristics, and Comparisons

In traditional production planning for make-to-stock environments, demand is forecast periodically and jobs are then scheduled efficiently, but without considering the impact of scheduling on future demand.

Here, we evaluate the typical improvement in profit provided by the optimal algorithms described in §3.1 relative to this traditional approach. Our assumptions about review points and when demand occurs are the same as in §2.2. Clearly, Algorithms DP-MTS1 and DP-MTS2 given in §3.1 that solve the general Problems MTS1 and MTS2 can solve the problems under this assumption.

In order to provide a comparison between the traditional and proposed approaches that is not confounded by capacity issues, we make the following assumption.

ASSUMPTION 2. *The production capacity in each time interval $[R_i, R_{i+1}]$, for $i = 0, \dots, m-1$, is sufficient that, using the traditional approach, all the forecast demand at review point R_{i+1} can be satisfied within that time interval alone.*

The traditional production planning algorithm (Nahmias 1997) for Problems MTS1 and MTS2 that is now described, Algorithm TR-MTS, forecasts expected demand and then tries to meet that demand plus a safety stock at minimum cost.

ALGORITHM TR-MTS.

At each review point R_i , for $i = 0, \dots, m-1$, do the following:

Step 1. For $j = 1, \dots, k$, forecast expected demand $e_{j,i+1}$ and standard deviation of the demand $\sigma_{j,i+1}$ at review point R_{i+1} from historical data, and find the number n_{ji} of type j jobs to produce in the time period $[R_i, R_{i+1}]$.

Step 2. Schedule n_{ji} type j jobs, for $j = 1, \dots, k$, in the time period $[R_i, R_{i+1}]$ to minimize total cost.

We consider three commonly used methods (Black 1997): moving average, simple exponential smoothing, and double exponential smoothing, for demand forecasting in Step 1. Let D_{ju} be the observed demand for type j jobs at time R_u . Using these methods, the demand forecast $e_{j,i+1}$ at review point R_{i+1} is, respectively, as follows: (i) $e_{j,i+1} = [\sum_{u=i-l+1}^i D_{ju}/l]$ for some integer l ; (ii) $e_{j,i+1} = [\rho D_{ji} + (1-\rho)e_{ji}]$ for some parameter $\rho \in (0, 1)$; (iii) $e_{j,i+1} = e_{j,i+1}^1 + e_{j,i+1}^2$, where $e_{j,i+1}^1 = [\rho D_{ji} + (1-\rho)e_{ji}]$ is the exponentially smoothed forecast of base demand, and $e_{j,i+1}^2 = [\theta(e_{j,i+1}^1 - e_{ji}^1) + (1-\theta)e_{ji}^2]$ is the exponentially smoothed forecast of trend, where $\rho, \theta \in (0, 1)$.

The value $\sigma_{j,i+1}$ in Step 1 is calculated as the standard deviation of the observed demands so far, i.e., $D_{j1}, \dots, D_{ji}$. We assume that the demand is normally distributed. Once the demand is forecast in Step 1, we calculate $n_{ji} = \max\{0, e_{j,i+1} + [2.05\sigma_{j,i+1}] - I_{ji}\}$, where $[2.05\sigma_{j,i+1}]$ is the added safety stock (corresponding to meeting 98% of demand if the actual demand is normally distributed), and I_{ji} is the inventory of type j jobs at time R_i after the demand for type j jobs has been satisfied or lost. In Step 2, the total inventory cost is minimized by processing the jobs in nonincreasing order of p_j/h_j without inserted idle time, such that the completion time of the last job is R_{i+1} (Smith 1956). In our computational study below, Algorithm TR-MTS is run once under the moving average forecasting method and with varying values of ρ and θ under the other two methods. The best solution from all those runs is used as the solution of TR-MTS.

Algorithm TR-MTS is described as a basis for comparison with our optimal algorithms, DP-MTS1 and DP-MTS2. However, the latter two algorithms may be difficult to implement and may require computation times that are too large for some applications. Therefore, we also propose simple greedy heuristics, denoted by GH-MTS1 and GH-MTS2, for Problems MTS1 and MTS2, respectively, that consider demand as a function of inventory. These heuristics use a greedy rule at each iteration to choose a job type and a number of jobs of this type to process next, such that the profit increase per time unit is maximized if the chosen jobs are processed next.

HEURISTIC GH-MTS1.

In each time interval $[R_{i-1}, R_i]$, for $i = 0, \dots, m$, do the following:

Step 0. Let P denote the total processing time of the jobs scheduled in the interval, where initially $P = 0$. Let I_j denote the inventory of type j jobs, where initially I_j is determined by the schedule already fixed up to time R_{i-1} . Let S denote the set of jobs already scheduled in the interval, where initially $S = \emptyset$.

Step 1. For $j = 1, \dots, k$, let u_j be the smallest integer such that $d_{j,R_i}(I_j + u_j) \geq d_{j,R_i}(I_j) + 1$. For $j = 1, \dots, k$, define

$$n_j = \begin{cases} u_j & \text{if } u_j p_j \leq \tau - P \\ 1 & \text{if } u_j p_j > \tau - P \text{ and } p_j \leq \tau - P \\ 0 & \text{otherwise.} \end{cases}$$

Let δ_j denote the increase in total profit if n_j type j jobs are added to S and all the jobs in S are scheduled in the interval $[R_i - P - n_j p_j, R_i]$ in nonincreasing order of p_j/h_j . Define the profit increase per time unit associated with job type j as $w_j = \delta_j/(n_j p_j)$ if $n_j > 0$, and 0 otherwise.

Step 2. Choose job type l such that $w_l = \max\{w_j > 0 \mid j = 1, \dots, k\}$. If no such l exists, then move to the next time interval, update $i = i + 1$, and go to Step 0. Otherwise, add n_l type l jobs to S and schedule the jobs in S in the interval $[R_i - P - n_l p_l, R_i]$ in nonincreasing order of p_j/h_j . Set $I_l = I_l + n_l$ and $P = P + n_l p_l$. Go to Step 1.

In Step 1 of Heuristic GH-MTS1, u_j is the smallest increment in production that will stimulate additional demand. Also, n_j is the number of type j jobs out of u_j that can be scheduled in the remaining processing time in the interval $[R_{i-1}, R_i]$. Then, w_j denotes the “bang-for-buck” ratio of increased profit to additional scheduling time used for job type j . In Step 2, l denotes the job with the highest bang-for-buck ratio that can schedule at least one unit within the interval. If l exists, then n_l jobs of type l are scheduled within the interval, backward from time R_i to R_{i-1} . Since $n_l = 0$ at the next iteration, job type l can not be selected again within the same interval.

HEURISTIC GH-MTS2.

In each time interval $[R_{i-1}, R_i]$, for $i = 1, \dots, m$, do the following:

Step 0. Let t denote the current time, where initially $t = R_{i-1}$. Let I_j denote the inventory of type j jobs at time t and V_j the average inventory of type j jobs up to time t . Initially, I_j and V_j are determined by the schedule up to time R_{i-1} .

Step 1. For $j = 1, \dots, k$, let u_j be the smallest integer such that $u_j p_j \leq R_i - t$, and

$$d_{j, R_i} \left(\frac{V_j t + \sum_{x=0}^{u_j-1} (I_j + x) p_j + (I_j + u_j)(R_i - t - u_j p_j)}{R_i} \right) \\ \geq d_{j, R_i} \left(\frac{V_j t + I_j(R_i - t)}{R_i} \right) + 1.$$

For $j = 1, \dots, k$, define

$$n_j = \begin{cases} u_j & \text{if such a } u_j \text{ exists} \\ 1 & \text{if no such } u_j \text{ exists and } p_j \leq R_i - t \\ 0 & \text{otherwise.} \end{cases}$$

Let δ_j denote the increase in total profit if n_j type j jobs are scheduled in the interval $[t, t + n_j p_j]$. Define the profit increase per time unit associated with job type j as $w_j = \delta_j/(n_j p_j)$ if $n_j > 0$, and 0 otherwise.

Step 2. Choose job type l such that $w_l = \max\{w_j > 0 \mid j = 1, \dots, k\}$. If no such l exists, then schedule idle time during $[t, t + 1]$, and set $t = t + 1$ and $V_j = (V_j t + I_j)/(t + 1)$. Otherwise, schedule n_l type l jobs in the interval $[t, t + n_l p_l]$, and update $I_l = I_l + n_l$, $V_l = [V_l t + \sum_{x=0}^{n_l-1} (I_l + x) p_l]/(t + n_l p_l)$, $V_j = [V_j t + I_j(n_l p_l)]/(t + n_l p_l)$ for $j = 1, \dots, k$ and $j \neq l$, and $t = t + n_l p_l$. If $t = R_i$, then move to the next time interval, set $i = i + 1$, and go to Step 0. Otherwise, go to Step 1.

In Step 1 of GH-MTS2, the definitions of u_j and n_j are similar to those in GH-MTS1 but use the different threshold for an increase in demand specified by Problem MTS2. If n_l jobs of type l are scheduled in Step 2, then the average inventory of all job types is recomputed. GH-MTS2 differs in two significant ways from GH-MTS1: (i) In each time interval $[R_{i-1}, R_i]$, jobs are scheduled forward from R_{i-1} to R_i . (ii) At each decision time t , it is possible to schedule an idle time period from t to $t + 1$.

Our computational experiments on GH-MTS1 and GH-MTS2 address the following questions: (i) How large is the relative improvement from the total profit generated by TR-MTS, compared to the optimal total profit? (ii) How large is the relative improvement from the total profit generated by TR-MTS, compared to the total profit provided by the greedy heuristics? (iii) How do these relative improvements vary with problem parameters? (iv) What other solution characteristics are changed by considering revenue?

We use the following general demand function of the inventory level or average inventory level x : $d_{jt}(x) = \lceil \alpha_{jt} x^{\beta_{jt}} \rceil$, where $\alpha_{jt} > 0$ and $0 < \beta_{jt} < 1$, for job type $j = 1, \dots, k$, and time $t \in \{R_1, \dots, R_m\}$. This demand function is a discretized version of the continuous demand function $d(x) = \alpha x^\beta$ commonly used in the inventory literature (Baker and Urban 1988a, b; Wang and Gerchak 2001; Balakrishnan et al. 2004). Such demand functions are nondecreasing, representing the fact that demand is stimulated by inventory.

Preliminary testing showed that the number of job types k , job processing times p_j , initial inventory I_j^0 , and unit salvage values s_j have no significant impact on the relative improvement provided

by our approach. Therefore, our experimental design considers ranges only for m , τ , r_j (with h_j fixed), α_{jt} , and β_{jt} . Since Algorithms DP-MTS1 and DP-MTS2 may require excessive computational times for some problems with $k \geq 3$, we consider problems with $k \in \{1, 2\}$. Our parameter settings follow:

- (a) The number of review points $m \in \{5, 10, 20\}$, and the length of each review period $\tau \in \{25, 50, 100\}$.
- (b) Processing times of jobs p_j are integers from $U[1, \dots, 8]$.
- (c) Unit job revenues r_j are integers from $U[400, \dots, 600]$, $U[800, \dots, 1,200]$, or $U[1,600, \dots, 2,400]$.
- (d) The initial inventory at time 0 is $I_j^0 = 0$, and unit inventory holding costs per unit time of jobs h_j and unit salvage values of jobs s_j are integers from $U[1, \dots, 3]$ and $U[10, \dots, 100]$, respectively.
- (e) Demand function parameters α_{jt} and β_{jt} are both generated from $U[0.4, 0.6]$, $U[0.6, 0.8]$, or $U[0.8, 1.0]$.

Since Algorithm TR-MTS considers only holding cost but our proposed algorithms also consider revenue, the range of $h_j T / r_j$ values generated is critical to the reasonableness of our results. The range we use is between 5% and 1,500%, with a median value of 100%. If T represents one year, this range of $h_j T / r_j$ includes the typical range, 9% to 50%, commonly used in the literature (Bowersox et al. 2002, p. 291) without using excessively large revenue parameters relative to holding cost parameters.

For each combination of the six parameters ($k, m, \tau, r_j, \alpha_{jt}, \beta_{jt}$), we test 10 randomly generated problem instances, for a total of $2 \times 3 \times 3 \times 3 \times 3 \times 10 = 4,860$ instances. Algorithm DP-MTS2 is not capable of solving every test problem within 30 CPU minutes, therefore we replace it by a simplified version, DP-MTS2', which aggregates all the states with the same $(I_1, \dots, I_k)$ and similar $(V_1, \dots, V_k)$ values into one state. The total profit generated by DP-MTS2' is a lower bound on that generated by DP-MTS2. In TR-MTS, we initiate all three forecasting methods using actual demand from DP-MTS1 or DP-MTS2' at the first few review points. In the moving average method, the demand at every other review point is forecast to be the mean of the realized demand at the three previous review points. In the simple and double exponential smoothing methods, we let ρ and θ be 0.2, 0.5, or 0.8. To compute the profit that can be

generated by TR-MTS for a test instance, TR-MTS is run 13 times for the test instance: once for the moving averages, three times for simple exponential smoothing corresponding to the three different ρ values, and nine times for double exponential smoothing corresponding to the nine combinations of ρ and θ values. The best solution from the 13 runs of TR-MTS is used as the solution of TR-MTS.

Each row in Table 4 shows mean results over 180 random test problems—10 for each of the 18 combinations of $(k, \alpha_{jt}, \beta_{jt})$ with the same (m, τ, r_j) combination. Each row in Table 5 shows mean percentage results over 540 test problems—10 for each of the 54 combinations of (k, m, τ, r_j) with the same $(\alpha_{jt}, \beta_{jt})$ combination. Columns 4 and 8 in Table 4 and Columns 3 and 7 in Table 5 show the relative improvement from TR-MTS to DP-MTS1 and from TR-MTS to DP-MTS2', defined by $(F_{DP1} - F_{TR})/F_{TR}$ and $(F_{DP2'} - F_{TR})/F_{TR}$, respectively, where F_{TR} , F_{DP1} , and $F_{DP2'}$ denote the optimal profit in the solutions generated by TR-MTS, DP-MTS1, and DP-MTS2', respectively. Since the optimal profit in DP-MTS2' is a lower bound on the optimal profit in DP-MTS2, the relative improvement from TR-MTS to DP-MTS2' is a lower bound on that from TR-MTS to DP-MTS2. Columns 5 and 9 in Table 4 and Columns 4 and 8 in Table 5 show the relative improvements in net profit from TR-MTS to Heuristics GH-MTS1 and GH-MTS2, defined by $(F_{GH1} - F_{TR})/F_{TR}$ and $(F_{GH2} - F_{TR})/F_{TR}$, respectively, where F_{GH1} , F_{GH2} , and F_{TR} are the total profits of the solutions generated by GH-MTS1, GH-MTS2', and TR-MTS, respectively. We also report the capacity utilization of TR-MTS, DP-MTS1, and DP-MTS2', which is defined as the percentage of the total available time T during which the machine is processing jobs, in Columns 6, 7, 10, and 11 in Table 4, and Columns 5, 6, 9, and 10 in Table 5. For problem MTS1, the results show that, on average, our dynamic programming algorithm DP-MTS1 provides a 131.1% improvement in profit and our greedy Heuristic GH-MTS1 provides a 111.3% improvement. Similarly, for Problem MTS2, DP-MTS2' provides a 199.7% improvement in profit and GH-MTS2 provides a 178.8% improvement. In both cases, it is encouraging from a practical viewpoint that a large percentage of the improvement achieved by the optimal dynamic programming algorithm is

Table 4 Comparison of Traditional Approaches, Optimal Algorithms, and Greedy Heuristics for Make-to-Stock Problems

<i>m</i> (1)	τ (2)	r_j (3)	Problem MTS1				Problem MTS2			
			Relative improvement (%)		Capacity utilization (%)		Relative improvement (%)		Capacity utilization (%)	
			DP-MTS1 (4)	GH-MTS1 (5)	TR-MTS (6)	DP-MTS1 (7)	DP-MTS2' (8)	GH-MTS2 (9)	TR-MTS (10)	DP-MTS2' (11)
5	25	[400, 600]	77.8	71.5	45.2	95.8	160.4	147.3	19.5	94.9
		[800, 1,200]	79.6	71.9	45.2	95.9	167.3	154.6	19.5	94.9
		[1,600, 2,400]	81.8	73.5	45.2	95.9	162.5	151.0	19.5	94.9
5	50	[400, 600]	101.9	94.1	29.8	88.6	165.6	152.3	13.4	76.2
		[800, 1,200]	109.7	100.1	29.8	93.4	190.1	173.7	13.4	82.1
		[1,600, 2,400]	115.0	110.4	30.0	95.4	205.7	200.5	13.4	91.5
5	100	[400, 600]	103.9	82.1	22.8	65.8	135.1	111.8	9.6	46.7
		[800, 1,200]	116.4	97.2	24.1	79.0	155.3	147.6	10.4	59.7
		[1,600, 2,400]	134.2	130.7	24.1	88.0	182.6	179.1	10.7	70.6
10	25	[400, 600]	102.8	92.1	40.4	94.6	169.2	154.3	20.0	82.3
		[800, 1,200]	108.1	93.5	40.4	96.6	195.2	179.2	20.0	89.5
		[1,600, 2,400]	114.0	94.3	40.6	97.0	205.1	189.8	20.0	93.8
10	50	[400, 600]	125.0	107.2	27.9	81.2	186.4	161.5	12.8	58.2
		[800, 1,200]	143.1	128.5	27.4	90.2	224.0	213.8	12.8	69.9
		[1,600, 2,400]	168.8	143.4	28.0	95.0	255.8	238.9	12.8	80.2
10	100	[400, 600]	112.4	64.7	19.0	57.4	123.4	69.0	8.3	37.0
		[800, 1,200]	139.2	114.9	19.9	71.8	164.2	158.7	8.3	46.0
		[1,600, 2,400]	182.9	172.3	20.5	83.6	225.2	220.4	8.5	60.1
20	25	[400, 600]	125.2	114.7	44.6	92.8	193.6	179.3	18.0	74.8
		[800, 1,200]	134.4	116.2	44.8	97.3	231.0	212.6	18.0	83.3
		[1,600, 2,400]	142.2	108.6	44.9	98.4	247.3	217.4	18.0	90.9
20	50	[400, 600]	140.7	110.0	28.6	78.2	169.2	115.2	12.5	54.0
		[800, 1,200]	168.7	151.7	28.6	87.2	231.4	227.0	12.8	62.7
		[1,600, 2,400]	208.4	192.6	28.6	94.1	303.5	295.5	12.8	71.9
20	100	[400, 600]	116.2	49.8	16.4	54.1	160.6	72.9	8.7	41.2
		[800, 1,200]	172.9	132.1	16.8	68.8	240.3	181.3	8.7	48.7
		[1,600, 2,400]	213.2	188.2	17.2	80.6	342.8	323.9	8.7	56.1

also achieved by the heuristic. Also of interest is that average capacity utilization improvements from 30.8% to 85.8% are observed in Problem MTS1, and from 13.7% to 70.8% in Problem MTS2.

Table 4 shows that the relative improvements for both problems do not vary monotonically with the number of review points. Increases in the length of the planning horizon tend to be associated with slight increases in relative improvement for both problems. Not surprisingly, larger job revenues are associated with significant increases in relative improvement. Capacity utilization in both the profit maximization and traditional models tends to decrease as the number of review points increases.

Table 5 shows that, in Problem MTS1, the relative improvements offered by DP-MTS1 and GH-MTS1 are

largest at an intermediate value of β_{jt} in most cases. However, the capacity utilization of TR-MTS increases with these parameters, and, more significantly, the capacity utilization of DP-MTS1 is more than 90% for larger values of the parameters. This result occurs because larger or more sensitive demand functions provide a larger incentive to hold inventory, which in turn tends to increase production and capacity utilization. In Problem MTS2, the relative improvements offered by DP-MTS2' and GH-MTS2 increase with α_{jt} and β_{jt} . The reason why this occurs here but not in Problem MTS1 is as follows: In general, larger values of α_{jt} and β_{jt} define a more sensitive demand function that tends to increase relative improvement. However, in the case of Problem MTS1, the use of the current inventory level to determine future demand

Table 5 Comparison of Traditional Approaches, Optimal Algorithms, and Greedy Heuristics for Make-to-Stock Problems

α_{jt} (1)	β_{jt} (2)	Problem MTS1				Problem MTS2			
		Relative improvement (%)		Capacity utilization (%)		Relative improvement (%)		Capacity utilization (%)	
		DP-MTS1 (3)	GH-MTS1 (4)	TR-MTS (5)	DP-MTS1 (6)	DP-MTS2' (7)	GH-MTS2 (8)	TR-MTS (9)	DP-MTS2' (10)
[0.4, 0.6]	[0.4, 0.6]	124.8	87.3	12.9	58.3	85.4	61.8	10.3	46.3
	[0.6, 0.8]	211.6	169.3	14.9	86.3	190.1	167.8	11.3	69.2
	[0.8, 1.0]	206.8	182.5	28.2	99.0	227.9	217.4	13.3	85.6
[0.6, 0.8]	[0.4, 0.6]	92.9	63.6	17.2	67.0	126.2	100.9	10.0	50.9
	[0.6, 0.8]	147.9	137.0	25.5	92.4	225.5	212.9	12.1	72.0
	[0.8, 1.0]	127.3	121.2	44.1	99.4	272.1	252.6	17.7	91.4
[0.8, 1.0]	[0.4, 0.6]	93.2	74.4	23.2	74.7	132.9	103.3	12.4	58.7
	[0.6, 0.8]	116.7	106.9	34.2	95.7	235.2	218.5	14.1	77.6
	[0.8, 1.0]	63.6	58.8	61.8	99.5	299.3	274.4	21.6	93.3

leads to unstable inventory fluctuations at higher levels of α_{jt} and β_{jt} . Since Problem MTS2 uses the more stable measure of average inventory level to compute demand, the effect on the relative improvement is monotonic.

In Algorithm TR-MTS, since jobs are scheduled as late as possible in each review period to minimize the total inventory holding cost, there is no inserted idle time between jobs in any review period. However, there may exist inserted idle time in a solution generated by DP-MTS2 or DP-MTS2'. In Problem MTS2, on the one hand, jobs should be processed as early as possible to provide a high average inventory level and thus stimulate demand. On the other hand, jobs should be processed as late as possible to minimize the inventory holding cost. This trade-off may lead to inserted idle time. Our results show that, in 84.1% of the test instances, some inserted idle time occurs in the solutions generated by DP-MTS2'; on average, the amount of inserted idle time is 4.5% of T .

Our last observation is that Algorithm TR-MTS utilizes a consistently lower percentage of capacity than the other algorithms, since it never produces more inventory than is needed in the current period. However, in an environment where demand is stimulated by inventory, even if the exact demand functions are not known, a potentially successful strategy is to produce more than the forecast amount. By comparison, the traditional approach typically leads to underproduction. We therefore conducted another set of tests on the following strategy, which is denoted

by TR-MTS-G. This strategy generalizes the traditional approach TR-MTS by considering to some extent future revenue implications when deciding how many units of a job type to produce in the current period. We therefore propose TR-MTS-G as a reasonable benchmark that might be used by a manager who recognizes the revenue-generating effect of carrying inventory. We assume that the demand is normally distributed. Approach TR-MTS-G has the same steps as TR-MTS, except that once the expected demand $e_{j,i+1}$ and standard deviation $s_{j,i+1}$ of each type j job at review point R_{i+1} are forecast as in Step 1 of TR-MTS, the anticipated demand at R_{i+1} is calculated to be $\lceil \gamma_j(e_{j,i+1} + 2.05\sigma_{j,i+1}) \rceil$, and the number of jobs to produce n_{ji} is defined to be $\max\{0, \lceil \gamma_j(e_{j,i+1} + 2.05\sigma_{j,i+1}) \rceil - I_{ji}\}$, where γ_j represents the degree of inventory stimulation on demand observed in the previous period, which is calculated as the ratio of the inventory of type j jobs at time R_i and the actual demand of type j jobs at R_i .

Table 6 shows the relative percentage improvement offered by the generalized approach TR-MTS-G over TR-MTS and the capacity utilization of TR-MTS-G. All the test problems used are the same as in Table 4. Although TR-MTS-G is a simple modification of the traditional approach TR-MTS, our results show that this strategy can improve the total profit by an average of 31.3% for Problem MTS1 and an average of 69.0% for Problem MTS2. Of course, these improvements are much smaller than the 111.3% and 178.8% average improvements found for Heuristics GH-MTS1 and GH-MTS2, respectively, in Table 4.

Table 6 Comparison of Approaches TR-MTS and TR-MTS-G for Make-to-Stock Problems

m (1)	τ (2)	r_j (3)	Problem MTS1		Problem MTS2	
			Relative improvement (%)	TR-MTS-G capacity utilization (%)	Relative improvement (%)	TR-MTS-G capacity utilization (%)
5	25	[400, 600]	24.1	59.1	59.0	49.3
		[800, 1,200]	25.3	58.4	63.7	49.2
		[1,600, 2,400]	26.4	58.6	66.5	48.7
5	50	[400, 600]	24.5	40.1	52.8	34.3
		[800, 1,200]	28.0	40.5	63.6	35.2
		[1,600, 2,400]	33.0	41.8	86.8	36.0
5	100	[400, 600]	18.4	33.0	31.5	19.3
		[800, 1,200]	25.4	35.6	34.1	22.1
		[1,600, 2,400]	32.3	36.8	58.1	24.7
10	25	[400, 600]	30.3	54.7	67.5	44.9
		[800, 1,200]	32.6	53.9	71.0	43.8
		[1,600, 2,400]	33.5	53.9	70.3	44.3
10	50	[400, 600]	30.3	38.5	57.1	28.0
		[800, 1,200]	36.0	38.2	71.2	28.6
		[1,600, 2,400]	40.9	39.1	77.2	28.2
10	100	[400, 600]	23.4	27.9	53.6	20.1
		[800, 1,200]	33.7	29.6	68.1	21.9
		[1,600, 2,400]	40.8	31.2	89.7	23.9
20	25	[400, 600]	31.0	55.5	77.0	40.3
		[800, 1,200]	33.4	56.1	83.3	41.0
		[1,600, 2,400]	34.6	56.8	89.2	42.5
20	50	[400, 600]	30.2	35.5	69.9	33.8
		[800, 1,200]	36.5	35.8	80.7	33.8
		[1,600, 2,400]	40.1	36.2	88.7	34.2
20	100	[400, 600]	22.6	21.1	46.8	26.5
		[800, 1,200]	35.8	22.6	74.5	32.2
		[1,600, 2,400]	42.6	24.6	112.2	33.4

4. First-Mover Problems

In this section, we consider situations where first-mover advantage, or bringing a product to market before competitors do so, generates substantial revenue. A review of the literature related to first-mover advantage is provided by Lieberman and Montgomery (1988). Kerin et al. (1992) indicate that the advantages in eventual market share from being a first mover motivate companies to pursue preemptive strategies to achieve this status. In some cases, multiple products with first-mover advantage may compete for limited resources with other similar products, and with more established products with time-invariant revenue functions. Relevant examples include a drug company with several products in various stages of development, a software developer trying to meet several preannounced product release dates, and a defense contractor with multiple contracts.

The planning horizon $[0, T]$ is divided evenly into m intervals $[0, l], (l, 2l], \dots, ((m-l)l, ml]$, where l is the length of each interval and $T = ml$. The long-run revenue from completing x_{ij} jobs of type j in interval i is a general function $r_{ij}(x_{ij})$, which is positive, increasing in x_{ij} , and nonincreasing in i for any given $j = 1, \dots, k$ and $x_{ij} \geq 0$. This revenue function, which is nonincreasing over time, is not used to model higher prices that are charged by first movers than by followers. In fact, just the opposite is often the case. The revenue function models the fact that first mover status confers brand enhancement and thus increased long-run market share and revenue on first movers. The objective is to maximize the total revenue earned over the planning horizon. This problem is denoted by FM.

We first consider Problem FM with a fixed number of job types k . For ease of presentation, we consider an extended planning horizon $[0, 2T]$ with m

additional time intervals, $(T, T + l]$, $(T + l, T + 2l]$, $\dots$, $(T + (m - 1)l, 2T]$, where $r_{ij}(x) = -\infty$, for $x \geq 1$, $i = m + 1, \dots, 2m$, $j = 1, \dots, k$. We also define a dummy job type, 0, with $p_0 = T$ and $r_{i,0} = 0$ for $i = 1, \dots, 2m$.

There are two cases of problem FM to consider. In the first case, $p_{\min} \geq l$. It follows immediately that no more than one job can complete processing within a single time interval. Here, we need the following preliminary result:

LEMMA 1. *For any instance of Problem FM with $p_{\min} \geq l$, there exists an optimal schedule with no idle time before the completion time of the last job.*

PROOF. Consider a schedule with idle time of length I between two consecutive jobs. Consider a new schedule formed by starting the later job I time units earlier and removing the idle time. Since the revenue function is nonincreasing over time, the total revenue of the new schedule is no less than that of the original schedule. Repeating this process for all other idle time periods completes the proof. $\square$

Lemma 1 motivates the following backward dynamic programming algorithm for the first case of Problem FM.

ALGORITHM FM1.

Value Function

$F(v_0, \dots, v_k) =$ maximum revenue earned during the period $[\sum_{j=0}^k p_j v_j, 2T]$, given that at the start of that period v_j jobs of type j , for $j = 0, \dots, k$, have been processed.

Boundary Condition

$$F(v_0, \dots, v_k) = 0, \quad \text{for } T \leq \sum_{j=0}^k p_j v_j \leq 2T \text{ and} \\ 0 \leq v_j \leq \lfloor T/p_j \rfloor, \quad \text{for } j = 0, \dots, k.$$

Optimal Solution Value

$$F(0, \dots, 0).$$

Recurrence Relation

$$F(v_0, \dots, v_k) = \max_{0 \leq i \leq k} \{r_{[\sum_{j=0}^k p_j v_j + p_i]/l, i}(1) \\ + F(v_0, \dots, v_{i-1}, v_i + 1, v_{i+1}, \dots, v_k)\}.$$

In Algorithm FM1, any optimal schedule uses no idle time and completes processing at a time t , where

$T \leq t \leq 2T$, and t is also the completion time of a job of type 0. The recurrence relation adds one new job to the partial schedule. The optimality and running time of Algorithm FM1 are discussed after consideration of the second case of Problem FM.

In the second case of Problem FM, $p_{\min} < l$. Here, an optimal schedule may include idle time before the completion time of the last job, as demonstrated by the following example:

EXAMPLE 1.

$$k = 1, \quad p_1 = 3, \quad l = 4, \quad T = 8, \\ r_{11}(1) = r_{21}(1) = 1, \quad r_{11}(2) = r_{21}(2) = 3.$$

The best schedule that does not contain idle time before the last job completes jobs at times 3 and 6, for a total revenue of 2. However, an optimal schedule inserts idle time from 0 to 2, and completes jobs at times 5 and 8, for a total revenue of 3.

In view of Example 1, we need the following preliminary result:

LEMMA 2. *For any instance of Problem FM with $p_{\min} < l$, there exists an optimal schedule in which, following any period of idle time, the next job processed completes at some time $gl + 1$, where g is integer and $1 \leq g \leq 2m - 1$.*

PROOF. Consider a schedule that contains idle time of length I , followed by the processing of a job that completes at time $gl + t \leq (g + 1)l$, where g is integer, $1 \leq g \leq 2m - 1$, and $t \geq 2$. If $I \leq t - 1$, then start the job I time units earlier and remove the idle time. Alternatively, if $I > t - 1$, then start the job $t - 1$ time units earlier, which leaves some idle time; the job now satisfies the lemma. In both cases, the revenue of the schedule is unchanged. Repeating this process for all other idle time periods completes the proof. $\square$

Lemma 2 motivates the following algorithm for the second case of Problem FM.

ALGORITHM FM2.

Value Functions

$F(g, u) =$ maximum revenue earned during the period $[0, 2T]$ (if $g = 0$) or $[gl + 1 - p_u, 2T]$ (if $g \neq 0$), given that the first job processed in that time period is a type u job, for $u = 0, \dots, k$. $G(g, u, v_0, \dots, v_k) =$ maximum revenue earned by v_j type j jobs, for $j = 0, \dots, k$, given that they are processed consecutively without idle time during the period $[0, \sum_{j=0}^k v_j p_j]$ (if

$g = 0$) or $[gl + 1 - p_u, gl + 1 - p_u + \sum_{j=0}^k v_j p_j]$ (if $g \neq 0$), and the first of those jobs processed is a type u job, for $u = 0, \dots, k$.

Boundary Conditions

$$F(g, u) = 0, \quad \text{for } g \geq m \text{ and } 0 \leq u \leq k.$$

$$G(g, u, 0, \dots, 0) = 0, \quad \text{for } 0 \leq g \leq m \text{ and } 0 \leq u \leq k.$$

Optimal Solution Value

$$\max_{0 \leq u \leq k} \{F(0, u)\}.$$

Recurrence Relations

$$G(g, u, v_0, \dots, v_k)$$

$$= \begin{cases} \max_{(i, h_0, \dots, h_k) \in S_1} \left\{ G(g, u, v_0 - h_0, \dots, v_k - h_k) + \sum_{j=0}^k r_{ij}(h_j) \right\}, \\ \text{where } S_1 = \left\{ (i, h_0, \dots, h_k) \mid i = \left\lceil \left(\sum_{j=0}^k v_j p_j \right) / l \right\rceil, \right. \\ \quad 0 \leq (i-1)l - \sum_{j=0}^k (v_j - h_j) p_j \\ \quad < \max_{0 \leq j \leq k} \{p_j \mid h_j > 0\}, 0 \leq h_j \leq v_j, \\ \quad \left. \text{for } j = 0, \dots, k \right\} \\ \max_{(i, h_0, \dots, h_k) \in S_2} \left\{ G(g, u, v_0 - h_0, \dots, v_k - h_k) + \sum_{j=0}^k r_{ij}(h_j) \right\}, \\ \text{where } S_2 = \left\{ (i, h_0, \dots, h_k) \mid i = \left\lceil \left(gl + 1 - p_u + \sum_{j=0}^k v_j p_j \right) / l \right\rceil, \right. \\ \quad 0 \leq (i-1)l - gl - 1 + p_u - \sum_{j=0}^k (v_j - h_j) p_j \\ \quad < \max_{0 \leq j \leq k} \{p_j \mid h_j > 0\}, 0 \leq h_j \leq v_j, \\ \quad \left. \text{for } j = 0, \dots, k \right\}. \end{cases}$$

$$F(g, u)$$

$$= \begin{cases} \max_{\substack{g', u' \mid g' \geq g+1, 0 \leq u' \leq k, \\ \sum_{j=0}^k v_j p_j \leq g' l + 1 - p_{u'}}} \{F(g', u') + G(g, u, v_0, \dots, v_k)\}, & \text{if } g = 0 \\ \max_{\substack{g', u' \mid g' \geq g+1, 0 \leq u' \leq k, \\ \sum_{j=0}^k v_j p_j \leq (g'-g)l + p_u - p_{u'}}} \{F(g', u') + G(g, u, v_0, \dots, v_k)\}, & \text{if } g \neq 0. \end{cases}$$

In Algorithm FM2, jobs are scheduled backward in batches using the recurrence relations for $F(g, u)$. The last batch of jobs considered in the recurrence relations consists of h_j type j jobs, for $j = 0, \dots, k$. These jobs are processed consecutively without idle time in the period $[0, \sum_{j=0}^k v_j p_j]$ (if $g = 0$) or $[gl + 1 - p_u, gl + 1 - p_u + \sum_{j=0}^k v_j p_j]$ (if $g \neq 0$), and a type u job is processed first. The maximum revenue contributed by this last batch of jobs is $G(g, u, v_0 - h_0, \dots, v_k - h_k)$. The sets S_1 and S_2 define all feasible values for $i, h_0, \dots, h_k$ in the case $g = 0$ and the case $g \neq 0$, respectively.

THEOREM 9. When k is fixed, Problem FM is solvable in polynomial time.

PROOF. If $p_{\min} \geq l$, then we apply Algorithm FM1. From Lemma 1, FM1 compares the revenues of all potentially optimal state transitions. If $p_{\min} < l$, then we apply Algorithm FM2. From Lemma 2, FM2 compares the revenues of all potentially optimal state transitions. Thus, the application of either FM1 or FM2 provides an optimal schedule for Problem FM.

It requires $O(k)$ time to determine whether $p_{\min} \geq l$. Now, when $p_{\min} \geq l$, Algorithm FM1 uses $O((T/p_{\min})^{k+1})$ values for the state space. Each use of the recurrence relation requires $O(k)$ comparisons. Thus, the overall time complexity of FM1 is $O(k(T/p_{\min})^{k+1})$. The size of the encoding of an instance of this problem is $O(\log T + \log l + k \log p_{\max} + (kT/l)(l/p_{\min}) \log r_{\max})$ bits. Therefore, when k is fixed, the running time of FM1 is polynomial. When $p_{\min} < l$, in Algorithm FM2 all G value functions are calculated before the F recursion is initiated. There are $O(k(T/l)(T/p_{\min})^{k+1})$ values for the state space. Each use of the G recurrence relations requires $O((l/p_{\min})^{k+1})$ comparisons, since the value of i is uniquely specified by g, U , and $h_0, \dots, h_k$. Given all the G value functions, the remaining time required to calculate all the F value functions is less than the time required to calculate all the G value functions. Thus, the overall time complexity of FM2 is $O(k(T/l)(T/p_{\min})^{k+1}(l/p_{\min})^{k+1})$. Since $p_{\min} < l$ when FM2 is applied, we may write $O(T/l)$ as $O(T/p_{\min})$. Since by definition $l \leq T$, the overall time complexity of FM2 is no more than $O(k(T/p_{\min})^{2k+3})$. The size of the encoding of an instance of this problem is $O(\log T + \log l + k \log p_{\max} + (kT/l)(l/p_{\min}) \log r_{\max}) =$

$O(\log T + \log l + k \log p_{\max} + (kT/p_{\min}) \log r_{\max})$ bits. Therefore, when k is fixed, the running time of FM2 is polynomial. $\square$

We now show that, when k is arbitrary, Problem FM is solvable in pseudopolynomial time. Our algorithm first finds the maximum revenue that is achievable for the scheduling subproblems in which all jobs are completed within the same time intervals, where only the first job is allowed to start in an earlier time interval. Then, it uses dynamic programming to link together the optimal solutions to these subproblems.

ALGORITHM FMA.

Single Interval Subproblem

For $j = 1, \dots, k$ and $p_{\min} \leq t_1 \leq t_2 \leq T$, compute $G(j, t_1, t_2) = \text{maximum revenue between times } t_1 - p_j \text{ and } t_2$, where $t_1 \leq t_2$ and $\lceil t_1/l \rceil = \lceil t_2/l \rceil$, given that at time t_1 , the number of jobs so far completed in the $\lceil t_1/l \rceil$ th time interval is one for type j and zero for all other job types.

Value Function

$F(t) = \text{maximum revenue earned during the period } [0, t]$, given that the last job which completes processing in the $\lceil t/l \rceil$ th time interval does so at time t or earlier.

Boundary Condition

$$F(0) = 0.$$

Optimal Solution Value

$$\max\{F(t) \mid T - p_{\min} + 1 \leq t \leq T\}.$$

Recurrence Relation

$$F(t) = \max\{G(i, t'_1, t) + F(t') \mid 1 \leq i \leq k, t' \leq t'_1 - p_i, \lceil t'_1/l \rceil = \lceil t/l \rceil, t' \leq (\lceil t/l \rceil - 1)l\}.$$

THEOREM 10. *Algorithm FMA finds an optimal schedule for Problem FM with an arbitrary number of job types in $O(k^2 T l^3 / p_{\min})$ time.*

PROOF. The single interval subproblem step computes the maximum revenue that is attainable for all state transitions in which all the jobs scheduled in the subproblem complete processing within the same time interval. The recurrence relation compares the revenue attained by connecting all possible single interval subproblems, allowing for the possibility

of idle time between time intervals. Since it compares the total revenue of all possible state transitions, Algorithm FMA finds an optimal schedule.

The single interval subproblem requires solution for $O(kl^2)$ values inside each interval; there are $O(T/l)$ intervals, for a total of $O(kTl)$ repetitions. Each repetition requires the solution of a nonlinear Integer Knapsack problem, which has $O(k)$ stages for the jobs, $O(l)$ states for the remaining processing time within the interval, and requires $O(l/p_{\min})$ comparisons each time. Thus, this step requires $O(k^2 T l^3 / p_{\min})$ time. The recurrence relation is solved for $O(T)$ values of t . Each use of the recurrence relation requires $O(kl)$ comparisons, since, by definition, t' and t are in the same time interval. Therefore, the overall time complexity of the recurrence relation is $O(kTl)$, which is less than the time complexity of the single interval subproblem step. It follows that the overall time complexity of Algorithm FMA is $O(k^2 T l^3 / p_{\min})$. $\square$

The following result shows that an algorithm that is as efficient as FMA is unlikely to exist for the problem with arbitrary k .

THEOREM 11. *The recognition version of Problem FM when k is arbitrary is binary NP-complete.*

PROOF. Consider an instance of Problem FM with only one time interval, i.e., $T = l$, and a linear revenue function $r_{1j}x_{1j}$. This problem is exactly equivalent to Integer Knapsack (Garey and Johnson 1979, p. 247). $\square$

Since the encoding of an instance of the problem requires

$$O(\log T + \log l + k \log p_{\max} + (kT/l)(l/p_{\min}) \log r_{\max}) \\ = O(\log T + \log l + k \log p_{\max} + (kT/p_{\min}) \log r_{\max})$$

bits, Algorithm FMA runs in pseudopolynomial time. Moreover, in view of Theorem 11, this is the best type of result possible, unless $P = NP$.

5. Concluding Remarks

On the research side, this paper proposes a refocusing of the classical scheduling literature, away from cost minimization and toward the consideration of revenue and net profit maximization objectives. We consider make-to-order environments where demand is stimulated by efficient scheduling performance,

which enhances customer goodwill. Here, we describe efficient optimal scheduling algorithms for maximizing net profit with the help of this demand stimulation and provide intractability results. Similar results are provided for make-to-stock environments, where the presence of inventory can stimulate demand in various ways. For both environments, we conduct computational studies that show that the benefits of considering the revenue and net profit implications of scheduling decisions are typically considerable. We describe simple heuristics that are easy to implement and that provide most of the benefit of our optimal algorithms. We also provide practical insights and sensitivity analysis based on our results. Finally, we describe efficient optimal algorithms and related intractability results for an environment where efficient scheduling brings products to market early, thus resulting in the achievement of a larger long-run market share. On the practical side, the paper provides simple tools that managers can use to make more efficient scheduling decisions and thus increase demand, revenue, and profit. Several of our proposed procedures are simple enough for spreadsheet implementation. Our computational experience suggests that the benefits of using our scheduling procedures should be robust and significant.

Several important issues remain open for future research. First, the solvability of the following two cases of Problem MTO2 needs to be resolved: the case with $L_j = p_j$ and an arbitrary number of job types k , and the case with general L_j and fixed k . Second, while the optimal algorithms that we describe are formally efficient, they are difficult to implement for problems with a large number of job types. Therefore, additional research is needed to find algorithms with improved computation time. Third, it would be valuable to design and test optimal enumerative algorithms for the problems that we identify as intractable. Fourth, research is needed to identify and evaluate additional factors that increase or decrease the benefits of considering the revenue and net profit implications of scheduling decisions. Fifth, other mechanisms by which efficient scheduling stimulates demand in make-to-order and make-to-stock environments should be studied. Finally, it is important to study practical cases of our first-mover models in particular industries.

We hope that our work will stimulate further research that extends the classical scheduling literature to consider the revenue and net profit implications of scheduling decisions. Besides offering the possibilities for improved decision making that we discuss, further research will help to unify the classical literature on scheduling with the newly developing literature on demand chain management. We also hope that our work will encourage practicing managers to view the scheduling function as having more strategic value than simply managing shop floor congestion and meeting due dates.

References

- Baker, R. C., T. L. Urban. 1988a. A deterministic inventory model with an inventory-level-dependent demand. *Naval Res. Logist.* **35** 99–116.
- Baker, R. C., T. L. Urban. 1988b. A deterministic inventory system with an inventory-level-dependent demand rate. *J. Oper. Res. Soc.* **39** 823–831.
- Balakrishnan, A., M. S. Pangburn, E. Stavroulaki. 2004. “Stack them high, let ‘em fly”: Lot-sizing policies when inventories stimulate demand. *Management Sci.* **50** 630–644.
- Berman, P., B. Dasgupta. 2000. Multi-phase algorithms for throughput maximization for real-time scheduling. *J. Combin. Optim.* **4** 307–323.
- Black, K. 1997. *Business Statistics: Contemporary Decision Making*, 2nd ed. West Publishing Company, St. Paul, MN.
- Blackburn, J. D. 1991. *Time-Based Competition: The Next Battle Ground in American Manufacturing*. Irwin/McGraw-Hill, Boston, MA.
- Blattberg, R. C., S. A. Neslin. 1990. *Sales Promotion: Concepts, Methods and Strategies*. Prentice Hall, Englewood Cliffs, NJ.
- Bowersox, D. J., D. J. Closs, M. B. Cooper. 2002. *Supply Chain Logistics Management*. McGraw-Hill, New York.
- Brucker, P. 1998. *Scheduling Algorithms*, 2nd ed. Springer, Berlin, Germany.
- Christopher, M. 1998. *Logistics and Supply Chain Management: Strategies for Reducing Cost and Improving Service*, 2nd ed. Financial Times and Prentice Hall, London, UK.
- Davis, D. 1989. Beating the clock. *Electronic Bus.* **136** 21–28.
- Duenyas, I., W. Hopp. 1995. Quoting customer lead-times. *Management Sci.* **41** 43–57.
- Frohlich, M. T., R. Westbrook. 2002. Demand chain management in manufacturing and services: Web-based integration, drivers and performance. *J. Oper. Management* **20** 729–745.
- Garey, M. R., D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, CA.
- Gerchak, Y., Y. Wang. 1994. Periodic-review inventory models with inventory-level-dependent demand. *Naval Res. Logist.* **41** 99–116.
- Geunes, J., Z.-J. Shen, H. E. Romeijn. 2004. Economic ordering decisions with market choice flexibility. *Naval Res. Logist.* **51** 117–136.

- Hall, N. G., H. Kamoun, C. Sriskandarajah. 1997. Scheduling in robotic cells: Classification, two and three machine cells. *Oper. Res.* **45** 421–439.
- Johnson, M. E. 2001. Learning from toys: Lessons in managing supply chain risk from the toy industry. *California Management Rev.* **43** 106–124.
- Kerin, R. A., P. R. Varadarajan, R. A. Peterson. 1992. First-mover advantage: A synthesis, conceptual framework and research propositions. *J. Marketing* **56** 33–52.
- Lawler, E. L., J. M. Moore. 1969. A functional equation and its application to resource allocation and sequencing problems. *Management Sci.* **16** 77–84.
- Lee, H. L., S. Whang. 2001. Demand chain excellence: A tale of two retailers. *Supply Chain Management Rev.* **5** 40–46.
- Lieberman, M. B., D. B. Montgomery. 1988. First-mover advantages. *Strategic Management J.* **9** 941–958.
- Loparic, M., Y. Pochet, L. A. Wolsey. 2001. The uncapacitated lot-sizing problem with sales and safety stocks. *Math. Programming* **89** 487–504.
- Moore, J. M. 1968. An n job, one machine sequencing algorithm for minimizing the number of late jobs. *Management Sci.* **15** 102–109.
- Nahmias, S. 1997. *Production and Operations Analysis*, 3rd ed. Irwin, Chicago, IL.
- Ohno, T. 1988. *Toyota Production System: Beyond Large-Scale Production*. Productivity Press, Cambridge, MA.
- Palaka, K., S. Erlebacher, D. H. Kropp. 1998. Lead-time setting, capacity utilization, and pricing decisions under lead-time dependent demand. *IIE Trans.* **30** 151–163.
- Papadimitriou, C. H., K. Steiglitz. 1982. *Combinatorial Optimization: Algorithms and Complexity*. Prentice Hall, Englewood Cliffs, NJ.
- Pinedo, M. 2002. *Scheduling: Theory, Algorithms and Systems*, 2nd ed. Prentice Hall, Englewood Cliffs, NJ.
- Seidmann, A., A. Tenenbaum. 1994. Throughput maximization in flexible manufacturing systems. *IIE Trans.* **26** 90–100.
- Selen, W., F. Soliman. 2002. Operations in today's demand chain management framework. *J. Oper. Management* **20** 667–673.
- Shingo, S. 1989. *A Study of the Toyota Production System from an Industrial Engineering Viewpoint*, rev. ed. Productivity Press, Cambridge, MA.
- Smith, W. E. 1956. Various optimizers for single stage production. *Naval Res. Logist. Quart.* **3** 59–66.
- Soliman, F., M. Youssef. 2001. The impact of some recent developments in e-business on the management of next generation manufacturing. *Internat. J. Oper. Production Management* **21** 538–564.
- Spearman, M., R. Zhang. 1999. Optimal lead-time policies. *Management Sci.* **45** 290–295.
- Stock, J. R., D. M. Lambert. 2001. *Strategic Logistics Management*, 4th ed. McGraw-Hill/Irwin, New York.
- Wang, Y., Y. Gerchak. 2001. Supply chain coordination when demand is shelf-space-dependent. *Manufacturing Service Oper. Management* **3** 82–87.
- Wein, L. 1991. Due date setting and priority sequencing in a multi-class M/G/1 queue. *Management Sci.* **37** 363–374.
- Weiss, H. J., M. E. Gershon. 1993. *Production and Operations Management*, 2nd ed. Simon & Schuster, Needham Heights, MA.
- Whang, S. 1988. *Pricing Computer Services: Incentive, Information and Queueing Effects*. Unpublished doctoral dissertation, University of Rochester, Rochester, NY.
- Whitt, W. 1999. Improving service by informing customers about anticipated delays. *Management Sci.* **45** 192–207.
- Zinn, W., P. C. Liu. 2001. Consumer response to retail stockouts. *J. Bus. Logist.* **22** 49–71.